

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN



LAB ASSIGNMENT 1
DATA PREPROCESSING

DATA MINING AND ITS APPLICATIONS
CSC14004 - 23KHDL2

Sinh viên: 23127084 - Dương Thành Lộc
23127250 - Trần Hồng Phương
23127339 - Mai Tấn Đạt
23127493 - Võ Hoàng Thương

Giảng viên hướng dẫn: TS. Lê Hoài Bắc
ThS. Lê Nhật Nam
CN. Võ Nhật Tân

Hồ Chí Minh, Tháng 1, 2026

Mục lục

1	Tiền xử lý dữ liệu ảnh	1
1.1	Mô tả bộ dữ liệu	1
1.1.1	Các lớp phân loại	1
1.1.2	Kích thước và cấu trúc dữ liệu	2
1.1.3	Nguồn dữ liệu	2
1.1.4	Ứng dụng tiềm năng	2
1.2	Tải ảnh	2
1.3	Thay đổi kích thước ảnh	3
1.3.1	Kết quả và phân tích	3
1.4	Chuyển đổi ảnh xám (Grayscale Conversion)	4
1.5	Chuẩn hóa dữ liệu	6
1.5.1	Chi tiết thực hiện	6
1.5.2	Kết quả và phân tích	7
1.6	Phát hiện biên	9
1.6.1	Mục đích	9
1.6.2	Nguyên lý phát hiện cạnh	10
1.6.3	Chi tiết thực hiện	10
1.6.4	Kết quả và phân tích	11
2	Tiền xử lý dữ liệu bảng	13
2.1	Mô tả bộ dữ liệu	13
2.1.1	Nguồn dữ liệu	13
2.1.2	Kích thước và cấu trúc	13
2.1.3	Mô tả	14
2.1.4	Ứng dụng tiềm năng	14
2.2	Xử lý dữ liệu thiếu	14
2.2.1	Xác định loại Missing	14
2.2.2	Điền dữ liệu	18
2.3	Chuẩn hóa dữ liệu	20
2.3.1	Đánh giá dữ liệu	20
2.3.2	Chuẩn hóa	21
2.4	Mã hóa biến phân loại	23
2.4.1	Xác định biến phân loại cần mã hóa	23
2.4.2	Mã hóa	24
2.4.3	Xử lý các thuộc tính phân loại có số lượng phần tử cao	24
2.5	Lựa chọn đặc trưng	24
3	Tiền xử lý dữ liệu văn bản	26
3.1	Mô tả bộ dữ liệu	26
3.2	Tokenization	26
3.2.1	Lý do sử dụng	26
3.2.2	Triển khai	27
3.2.3	Kết quả	27

3.3	Stop-word Removal	28
3.3.1	Lý do sử dụng	28
3.3.2	Triển khai	28
3.3.3	Kết quả	29
3.4	Stemming và Lemmatization	30
3.4.1	Lý do sử dụng	30
3.4.2	Triển khai	30
3.4.3	Kết quả	31
3.5	Text Vectorization	31
3.5.1	Lý do sử dụng	31
3.5.2	Cơ sở toán học	31
3.5.3	Bag of Words (BoW)	32
3.5.4	TF-IDF	32
3.5.5	Word2Vec	33
3.5.6	Triển khai	33
3.5.7	Kết quả	34
4	Tiền xử lý dữ liệu chuỗi thời gian	36
4.1	Mô tả Dữ liệu	36
4.1.1	Temporal Data	36
4.1.2	Dataset sử dụng	36
4.1.3	Quy mô dữ liệu	36
4.1.4	Mô tả chi tiết	36
4.1.5	Tiềm năng sử dụng	37
4.1.6	Lựa chọn của chúng em	37
4.2	Kỹ thuật tiền xử lý áp dụng	38
4.2.1	Parsing Date and Time	38
4.2.2	Handling Time Gaps	39
4.2.3	Extracting Time-Based Features	42
4.2.4	Resampling Time Series Data	45
4.2.5	Lag Features	49
5	Đánh giá chung và thảo luận	52
6	Lời cảm ơn	53
7	Tài liệu tham khảo	54

1 Tiền xử lý dữ liệu ảnh

1.1 Mô tả bộ dữ liệu

Nhóm sử dụng bộ dữ liệu **Fashion-MNIST**, đây là bộ dữ liệu phổ biến trong các bài toán phân loại ảnh quần áo.

Mỗi mẫu dữ liệu là một ảnh xám (*grayscale*) kích thước 28×28 pixel. Khi lưu dưới dạng `.csv`, mỗi ảnh được biểu diễn bởi 785 cột, trong đó:

- Cột đầu tiên là **nhãn lớp** (label) từ 0 đến 9.
- 784 cột còn lại là giá trị pixel (0–255) của ảnh, tương ứng với $28 \times 28 = 784$ điểm ảnh.

Mỗi hàng trong file dữ liệu tương ứng với một ảnh và nhãn của ảnh đó.

1.1.1 Các lớp phân loại

Các nhãn trong bộ dữ liệu bao gồm:

- 0 – T-shirt/top
- 1 – Trouser
- 2 – Pullover
- 3 – Dress
- 4 – Coat
- 5 – Sandal
- 6 – Shirt
- 7 – Sneaker
- 8 – Bag
- 9 – Ankle boot

1.1.2 Kích thước và cấu trúc dữ liệu

Bộ dữ liệu gồm hai tập:

- Tập **train**: 60,000 ảnh
- Tập **test**: 10,000 ảnh

Trong phạm vi báo cáo này, nhóm sử dụng tập **test** để thực hiện các bước tiền xử lý, trực quan hóa và thử nghiệm thuật toán.

Mỗi ảnh có:

- Kích thước: 28×28
- Số kênh màu: 1 (grayscale)
- Giá trị pixel: $[0, 255]$

1.1.3 Nguồn dữ liệu

Bộ dữ liệu được tải từ Kaggle tại: [Fashion MNIST](#)

1.1.4 Ứng dụng tiềm năng

Fashion-MNIST thường được sử dụng trong:

- Phân loại hình ảnh (Image Classification)
- Thử nghiệm các thuật toán Machine Learning và Deep Learning
- So sánh hiệu năng các mô hình CNN
- Thực hành các kỹ thuật tiền xử lý ảnh như normalization, resizing, edge detection
- Trích xuất đặc trưng hình dạng (shape-based features)

1.2 Tải ảnh

Đầu tiên, nhóm sử dụng thư viện **Pandas** để đọc dữ liệu từ file `fashion-mnist_test.csv` thông qua hàm `read_csv()`. Lúc này kiểu dữ liệu đang là DataFrame của pandas. Sau đó, nhóm tách dữ liệu thành:

- **y**: chứa nhãn lớp của ảnh.
- **X**: chứa các giá trị pixel của ảnh sau khi loại bỏ cột *label*.

Các giá trị pixel được chuyển sang mảng **NumPy** bằng thuộc tính **values** để tương thích với các thư viện xử lý ảnh như OpenCV.

Tiếp theo, dữ liệu được **reshape** từ dạng vector 1 chiều (784 phần tử) về ma trận ảnh 2 chiều kích thước 28×28 . Sau bước này, tập dữ liệu ảnh có dạng tensor 3 chiều:

(số ảnh, 28, 28)

có thể trực tiếp sử dụng trong các bước tiền xử lý ảnh như hiển thị, thay đổi kích thước (resize), phát hiện cạnh,...

1.3 Thay đổi kích thước ảnh

Sau khi chuyển dữ liệu sang mảng NumPy và reshape về kích thước ảnh gốc 28×28 , nhóm tiến hành thay đổi kích thước ảnh nhằm quan sát ảnh ở độ phân giải cao hơn và so sánh các phương pháp nội suy khác nhau.

Cụ thể, nhóm sử dụng hàm `cv2.resize()` trong thư viện OpenCV để phóng to ảnh lên hai kích thước:

- 128×128
- 256×256

Hai phương pháp nội suy được thử nghiệm gồm:

- **INTER_LINEAR**: nội suy song tuyến tính, tạo ảnh mượt hơn bằng cách tính trung bình có trọng số của các pixel lân cận.
- **INTER_NEAREST**: nội suy láng giềng gần nhất, giữ nguyên giá trị pixel gần nhất nên ảnh sắc nét nhưng dễ bị răng cưa.

1.3.1 Kết quả và phân tích

Kết quả



Figure 1: Kết quả thay đổi kích thước với các phương pháp nội suy khác nhau

Kết quả trực quan cho thấy:

- **INTER_LINEAR** cho ảnh mịn và tự nhiên hơn khi phóng to.

- `INTER_NEAREST` vì giữ nguyên giá trị pixel gần nhất nên vẫn xuất hiện hiện tượng vỡ hình như ảnh gốc

Phân tích

Ảnh gốc của Fashion-MNIST có kích thước 28×28 pixel. Việc resize lên kích thước lớn hơn (ví dụ: 128×128 hoặc 224×224) có thể hữu ích trong một số trường hợp:

- Ảnh dễ quan sát trực quan hơn, thể hiện rõ hơn hình dạng và đường viền của các sản phẩm thời trang.
- Đảm bảo tương thích với các mô hình pretrained (ví dụ: VGG, ResNet yêu cầu đầu vào 224×224). [1]

Tuy nhiên, việc tăng kích thước bằng nội suy (interpolation) có những hạn chế sau:

- **Không tạo thêm thông tin mới:** Ảnh gốc chỉ chứa $28 \times 28 = 784$ pixel thông tin thật, trong khi ảnh $128 \times 128 = 16,384$ pixel — phần lớn là giá trị được nội suy dựa trên dữ liệu gốc.
- **Tăng chi phí tài nguyên:** Bộ nhớ cần thiết tăng khoảng **21 lần** ($16,384/784 \approx 21$), đồng thời thời gian xử lý các thuật toán (edge detection, normalization) cũng tăng tương ứng.

Về phương pháp nội suy, **INTER_LINEAR** (bilinear interpolation) cho kết quả mượt mà hơn so với **INTER_NEAREST** (nearest-neighbor). `INTER_NEAREST` chỉ sao chép giá trị pixel gần nhất nên ảnh có xu hướng bị răng cưa (blocky artifacts), trong khi `INTER_LINEAR` tính trung bình có trọng số từ các pixel lân cận giúp chuyển đổi mượt hơn. Từ kết quả quan sát trực quan, không có sự khác biệt đáng kể giữa ảnh 128×128 và 256×256 , do lượng thông tin gốc vẫn chỉ là 28×28 .

Kết luận: Trong phạm vi bài tập tiền xử lý dữ liệu này, nhóm quyết định giữ nguyên kích thước gốc 28×28 của Fashion-MNIST vì dựa trên phân tích sự đánh đổi giữa chi phí tài nguyên (tăng 21 lần) và lợi ích thu được (chỉ cải thiện khả năng trực quan, không thêm thông tin mới). Resize nên được cân nhắc khi có yêu cầu cụ thể như sử dụng pretrained model hoặc cần trực quan hóa chi tiết.

1.4 Chuyển đổi ảnh xám (Grayscale Conversion)

Cách chuyển đổi ảnh màu sang ảnh xám

Vì bộ dữ liệu hiện tại mặc định là ảnh xám nên trong trường hợp dữ liệu ban đầu là ảnh màu RGB, ảnh xám được tạo bằng cách kết hợp ba kênh màu theo công thức độ sáng (luminance). Thay vì lấy trung bình đơn giản, các kênh được gán trọng số khác nhau do mắt người nhạy cảm với màu xanh lá hơn các màu còn lại.

Công thức chuyển đổi phổ biến:

$$Gray = 0.299R + 0.587G + 0.114B$$

trong đó R, G, B lần lượt là giá trị của ba kênh màu. Kết quả thu được là một ảnh một kênh biểu diễn cường độ sáng. [2]

Bộ dữ liệu Fashion-MNIST mặc định đã ở dạng ảnh xám (grayscale), mỗi ảnh chỉ gồm một kênh cường độ sáng thay vì ba kênh màu RGB. Do đó, ở bước này nhóm chỉ thảo luận nhằm so sánh khả năng lưu trữ thông tin giữa ảnh màu và ảnh xám.

Trong các bài toán thị giác máy tính, việc chuyển đổi từ ảnh màu sang ảnh xám thường được áp dụng để:

- giảm số chiều dữ liệu (từ 3 kênh RGB xuống 1 kênh),
- giảm chi phí tính toán và bộ nhớ,
- tập trung vào thông tin hình dạng, biên và cấu trúc thay vì màu sắc.

Đối với Fashion-MNIST, các lớp quần áo chủ yếu được phân biệt dựa trên **hình dạng (shape)** và **đường viền (edges)**. Màu sắc không phải là đặc trưng quan trọng, do đó việc sử dụng ảnh xám vẫn giữ được hầu hết thông tin cần thiết cho quá trình phân loại.

Lợi ích:

- Giảm kích thước dữ liệu xuống còn $\frac{1}{3}$ so với ảnh RGB.
- Tăng tốc độ xử lý và huấn luyện mô hình.
- Đơn giản hóa các bước tiền xử lý như normalization và edge detection.

Hạn chế:

- Mất hoàn toàn thông tin màu sắc.
- Không phù hợp với các bài toán mà màu là đặc trưng chính.

Khi nào nên và không nên sử dụng ảnh xám

Nên sử dụng:

- Bài toán dựa trên shape/edge/texture.
- Dataset lớn cần tối ưu thời gian tính toán.
- Ví dụ: chữ viết tay, Fashion-MNIST,...

Không nên sử dụng:

- Khi màu sắc là đặc trưng quan trọng để phân biệt lớp.
- Ví dụ: nhận diện trái cây, phân loại hoa, phân loại thực phẩm.

Kết luận: Do tập dữ liệu đã là ảnh xám từ đầu, nên không có code đổi với phần này. Tuy nhiên, việc sử dụng ảnh xám giúp giảm đáng kể chi phí tính toán và vẫn đảm bảo giữ được các đặc trưng quan trọng như đường viền và cấu trúc hình học của quần áo.

Vì vậy, ảnh xám là lựa chọn phù hợp và hiệu quả cho bài toán phân loại Fashion-MNIST.

1.5 Chuẩn hóa dữ liệu

Trong bộ dữ liệu Fashion-MNIST, giá trị pixel ban đầu nằm trong khoảng $[0, 255]$. Khoảng giá trị lớn này có thể gây khó khăn khi đưa vào các mô hình machine learning do:

- Các thuật toán tối ưu (gradient descent) thường hoạt động tốt hơn với dữ liệu có thang đo nhỏ và đồng nhất.
- Nhiều hàm kích hoạt (sigmoid, tanh) có miền giá trị $[-1, 1]$ hoặc $[0, 1]$, nên chuẩn hóa về cùng thang đo giúp tận dụng tốt vùng tuyến tính của hàm, nơi gradient lớn nhất.

Vì vậy, nhóm tiến hành chuẩn hóa dữ liệu và so sánh ba kỹ thuật chuẩn hóa:

- Min-Max scaling về $[0, 1]$
- Min-Max scaling về $[-1, 1]$
- Standardization (zero mean, unit variance)

1.5.1 Chi tiết thực hiện

Sau khi thay đổi kích thước ảnh về 128×128 , các giá trị pixel được chuyển sang kiểu `float32` để thuận tiện cho các phép toán số thực. Nhóm tiến hành áp dụng ba phương pháp chuẩn hóa khác nhau bao gồm Min-Max scaling về $[0, 1]$, Min-Max scaling về $[-1, 1]$ và Standardization.

1. Min-Max scaling về $[0, 1]$ Giá trị pixel được đưa về khoảng $[0, 1]$ theo công thức:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

trong đó:

- x là giá trị pixel ban đầu,
- x_{min}, x_{max} lần lượt là giá trị nhỏ nhất và lớn nhất của ảnh.

2. Min-Max scaling về $[-1, 1]$ Từ ảnh đã chuẩn hóa về $[0, 1]$, dữ liệu tiếp tục được biến đổi tuyến tính sang khoảng $[-1, 1]$:

$$x'' = 2x' - 1$$

3. Standardization (zero mean, unit variance) Dữ liệu được chuẩn hóa theo trung bình và độ lệch chuẩn:

$$x_{std} = \frac{x - \mu}{\sigma}$$

trong đó:

- μ là giá trị trung bình của **toàn bộ tập dữ liệu**,
- σ là độ lệch chuẩn của **toàn bộ tập dữ liệu**.

Sau đó, các ảnh và histogram phân phối pixel được trực quan hóa bằng Matplotlib để so sánh sự thay đổi về thang đo và hình dạng phân phối giữa các phương pháp.

1.5.2 Kết quả và phân tích



Figure 2: So sánh trực quan ảnh sau chuẩn hóa

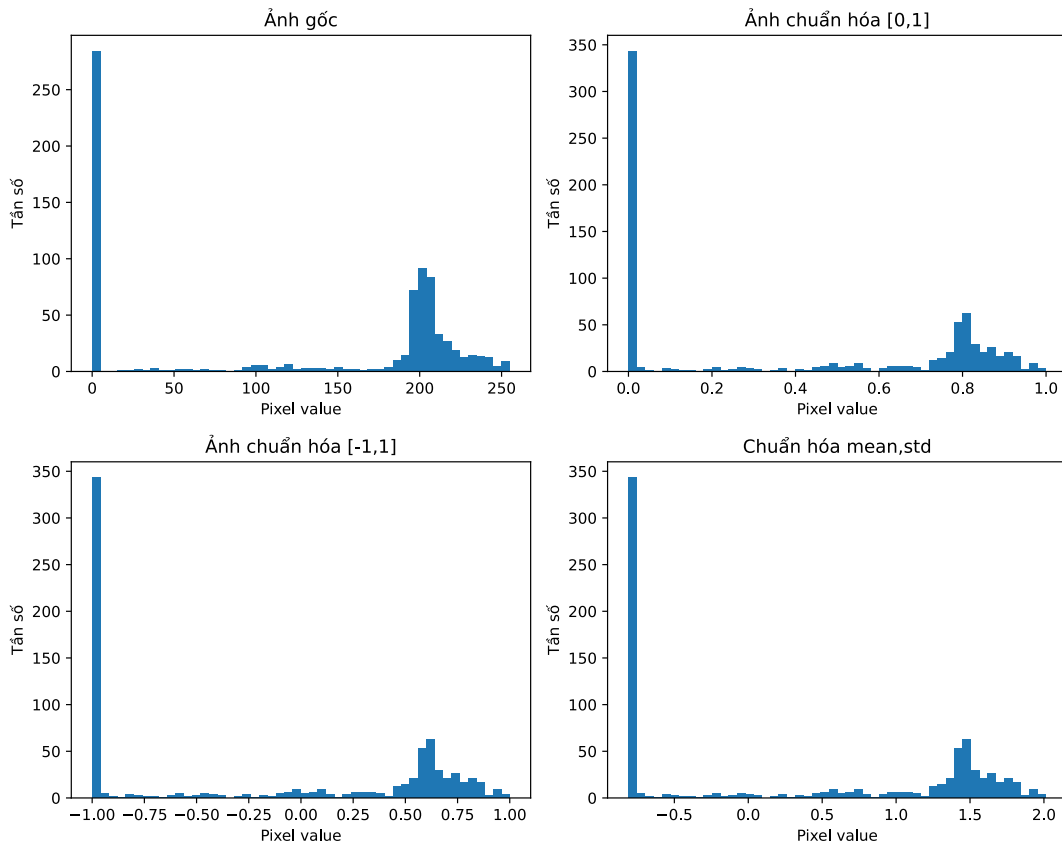


Figure 3: So sánh phân phối pixel

Quan sát ảnh và histogram thu được các nhận xét sau:

Ảnh gốc

- Pixel nằm trong khoảng $[0, 255]$
- Phần lớn giá trị tập trung gần 0 (nền đen) và một cụm lớn ở vùng sáng của quần áo
- Phân phối lệch, không cân bằng
- Thang đo lớn có thể làm gradient dao động mạnh khi huấn luyện

Chuẩn hóa $[0,1]$

- Chỉ thay đổi thang đo từ $[0, 255]$ về $[0, 1]$
- Hình dạng histogram gần như giữ nguyên, chỉ bị co lại khoảng giá trị pixel
- Giúp giảm biên độ giá trị $\rightarrow$ gradient nhỏ và ổn định hơn
- Đây thường là cách chuẩn hóa phổ biến nhất cho CNN do đơn giản, nhanh và đủ hiệu quả

Chuẩn hóa $[-1,1]$

- Giá trị được **center gần 0** hơn so với $[0,1]$
- Dữ liệu trong khoảng $[-1,1]$ chứa cả giá trị âm và dương, giúp đầu vào của mạng được phân bố đối xứng quanh 0. Điều này làm cho gradient trong quá trình backpropagation cân bằng hơn, tránh hiện tượng cập nhật trọng số lệch về một phía và giúp quá trình hội tụ ổn định hơn. Đặc biệt, khi sử dụng hàm kích hoạt **tanh** (có miền giá trị $[-1,1]$), việc chuẩn hóa về cùng thang đo giúp dữ liệu nằm trong vùng tuyến tính của hàm, nơi gradient lớn nhất, từ đó tăng tốc độ học của mô hình.

Standardization (mean = 0, std = 1)

- Standardization đặc biệt phù hợp với các thuật toán machine learning truyền thống như SVM, Logistic Regression và PCA. Các phương pháp này nhạy cảm với thang đo của đặc trưng do sử dụng khoảng cách Euclidean hoặc phương sai trong quá trình tối ưu. Nếu dữ liệu không được chuẩn hóa, các đặc trưng có biên độ lớn sẽ chi phối mô hình. Việc đưa dữ liệu về cùng thang đo (mean = 0, std = 1) giúp quá trình học ổn định hơn và đảm bảo các đặc trưng được đóng góp công bằng.

Kết luận

Fashion-MNIST là bộ dữ liệu ảnh xám phục vụ bài toán phân loại quần áo. Các đặc trưng quan trọng chủ yếu là hình dạng, biên và texture, nên các mô hình Convolutional Neural Networks (CNN) thường được lựa chọn vì khả năng khai thác thông tin cục bộ và học đặc trưng tự động hiệu quả hơn so với các thuật toán machine learning truyền thống.

Đối với CNN, việc chuẩn hóa giá trị pixel về khoảng $[0,1]$ là lựa chọn phù hợp nhất vì:

- Giữ nguyên cấu trúc và tương quan tương đối giữa các pixel, không làm biến dạng thông tin hình ảnh.
- Cách thực hiện rất đơn giản (chỉ cần chia cho 255 đối với Fashion-MNIST), chi phí tính toán thấp và dễ triển khai trên toàn bộ tập dữ liệu lớn.
- Tương thích tốt với hầu hết các kiến trúc CNN hiện đại và các hàm kích hoạt phổ biến như ReLU, sigmoid.

Ngoài ra tài liệu tham khảo từ TensorFlow cũng tiên xử lý đối với bộ dữ liệu này về $[0,1]$. [3] Vì vậy, đối với bộ dữ liệu Fashion-MNIST, chuẩn hóa Min-Max về $[0,1]$ mang lại sự cân bằng tốt giữa hiệu quả (dựa trên lý thuyết và kinh nghiệm thực tế với các bộ dữ liệu tương tự) và chi phí tính toán, nên được chọn làm phương pháp tiên xử lý chính cho bộ dữ liệu này.

1.6 Phát hiện biên

1.6.1 Mục đích

Đối với dữ liệu hình ảnh, các cạnh (edges) thường biểu diễn sự thay đổi mạnh về cường độ pixel, tương ứng với ranh giới của vật thể. Trong bộ dữ liệu Fashion-

MNIST, thông tin quan trọng chủ yếu nằm ở đường viền quần áo, cổ áo, tay áo, đế giày và hình dạng tổng thể. Do đó, việc trích xuất đặc trưng biên giúp làm nổi bật cấu trúc hình dạng và giảm ảnh hưởng của vùng nền, từ đó hỗ trợ quá trình phân loại.

Nhóm tiến hành áp dụng ba thuật toán phát hiện biên phổ biến: Sobel, Prewitt và Canny để so sánh chất lượng cạnh thu được.

1.6.2 Nguyên lý phát hiện cạnh

Trong ảnh số, cạnh (edge) thường xuất hiện tại những vị trí mà giá trị cường độ pixel thay đổi đột ngột so với vùng lân cận. Nói cách khác, cạnh tương ứng với những điểm có độ biến thiên cường độ sáng lớn.

Về mặt toán học, sự thay đổi này có thể được mô tả bằng đạo hàm bậc nhất của ảnh. Nếu đạo hàm theo một hướng có giá trị lớn, điều đó cho thấy tại vị trí đó tồn tại sự chuyển tiếp mạnh giữa hai vùng sáng – tối, và đây chính là cạnh của vật thể.

Các toán tử Sobel và Prewitt ước lượng đạo hàm theo hai hướng chính (ngang và dọc) thông qua các kernel tích chập. Độ lớn gradient được tính từ hai đạo hàm này cho biết mức độ thay đổi cường độ tại mỗi pixel; giá trị gradient càng lớn thì khả năng pixel đó nằm trên cạnh càng cao.

Đối với Canny, ngoài việc tính gradient, thuật toán còn áp dụng thêm các bước làm mảnh cạnh (Non-Maximum Suppression) và ngưỡng kép (Double Threshold với Hysteresis) nhằm loại bỏ nhiễu và chỉ giữ lại những cạnh có ý nghĩa hình học. Nhờ đó, Canny cho kết quả cạnh mảnh, liên tục và rõ ràng hơn so với Sobel và Prewitt.

1.6.3 Chi tiết thực hiện

1. Sobel Sobel ước lượng đạo hàm bậc nhất theo hai hướng ngang và dọc:

$$G_x = I * S_x, \quad G_y = I * S_y$$

trong đó S_x, S_y là các kernel Sobel kích thước 3×3 .

Trong cài đặt, đạo hàm ngang và đạo hàm dọc được tính theo hàm `Sobel()` có sẵn trong thư viện OpenCV, sau đó

Độ lớn gradient được tính theo:

$$G = \sqrt{G_x^2 + G_y^2}$$

Giá trị lớn biểu thị sự thay đổi mạnh $\rightarrow$ cạnh.

2. Prewitt Prewitt tương tự Sobel nhưng sử dụng kernel đơn giản hơn:

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}, \quad S_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Cài đặt bằng phép tích chập thông qua hàm `filter2D` của OpenCV.

Sau đó tính độ lớn gradient tương tự Sobel.

3. Canny Canny là phương pháp nâng cao gồm nhiều bước:

- Làm mượt bằng Gaussian Blur
- Tính gradient
- Non-Maximum Suppression (làm mảnh cạnh)
- Double Threshold + Hysteresis

Được cài đặt thông qua hàm `cv2.Canny()` với:

- `threshold1 = 50`: ngưỡng yếu (weak)
- `threshold2 = 150`: ngưỡng mạnh (strong)

Chỉ giữ lại các cạnh mạnh hoặc cạnh yếu nối với cạnh mạnh. Ở đây lý do cho việc chọn ngưỡng 50 và 150 là vì trong tài liệu OpenCV khuyến khích chọn tỉ lệ upper:lower là 2:1 hoặc 3:1. [4]

1.6.4 Kết quả và phân tích

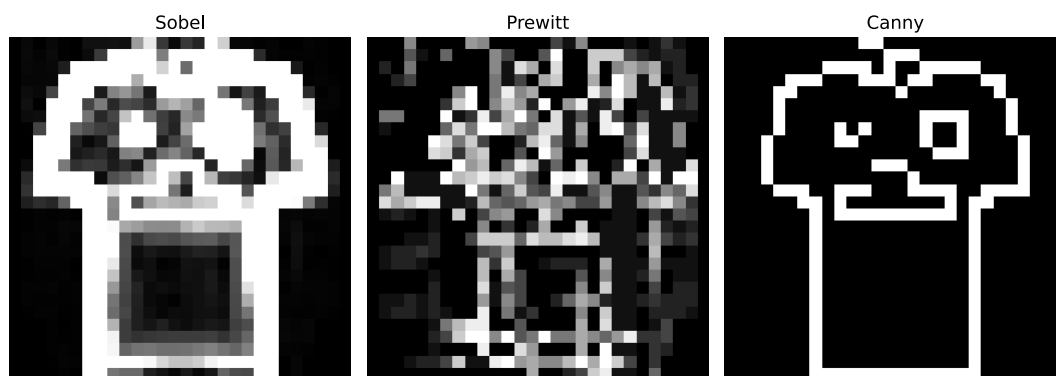


Figure 4: Ảnh minh họa kết quả 3 phương pháp trích xuất khác nhau

So sánh trực quan:

- **Sobel:** Làm nổi bật biên khá rõ nhưng vẫn giữ nhiều vùng nhiễu và biên dày.
- **Prewitt:** Ảnh dường như đã bị vỡ cấu trúc, biên rời rạc, đường viền không liên tục
- **Canny:** Biên rõ ràng, mảnh, liên tục hơn, giữ tốt hình dạng tổng thể của áo Nhiều nền được loại bỏ đáng kể nhờ bước Gaussian smoothing và Non-Maximum Suppression. Chỉ giữ lại các đường viền quan trọng của vật thể.

Ý nghĩa đối với Fashion-MNIST:

Các lớp quần áo trong Fashion-MNIST chủ yếu được phân biệt dựa trên **hình dạng và đường viền**. Do đó, việc trích xuất biên giúp:

- làm nổi bật cấu trúc hình học của vật thể (cổ áo, tay áo, đế giày,...),
- giảm ảnh hưởng của vùng nền và cường độ sáng,
- giảm số lượng đặc trưng cần học,
- giúp mô hình (đặc biệt CNN) tập trung vào shape thay vì texture dư thừa.

So với Sobel và Prewitt, Canny cung cấp đặc trưng biên sạch và rõ ràng hơn, do đó phù hợp hơn cho bước tiền xử lý hoặc làm kênh đầu vào bổ sung cho CNN.

2 Tiền xử lý dữ liệu bảng

2.1 Mô tả bộ dữ liệu

2.1.1 Nguồn dữ liệu

Bộ dữ liệu được lấy từ UCI - Adult. Tại: [Adult](#)

2.1.2 Kích thước và cấu trúc

Gồm 2 tập tin:

- **train:** 32561 dòng
- **test:** 16281 dòng

Trong bài, chúng em sử dụng train để thực hiện.

Số thuộc tính: 15

Thông tin thuộc tính:

- **age:** continuous.
- **workclass:** Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
- **textbfffnlwgt:** continuous.
- **education:** Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.
- **education-num:** continuous.
- **marital-status:** Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse.
- **occupation:** Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.
- **relationship:** Wife, Own-child, Husband, Not-in-family, Other-relative, Un-married.
- **race:** White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.

- **sex:** Female, Male.
- **capital-gain:** continuous.
- **capital-loss:** continuous.
- **hours-per-week:** continuous.
- **native-country:** United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinidad&Tobago, Peru, Hong, Holand-Netherlands.
- **income:** >50K, <=50K

2.1.3 Mô tả

Bộ dữ liệu được trích xuất từ cơ sở dữ liệu Điều tra dân số năm 1994, thể hiện thu nhập của một người nhỏ hơn hay lớn hơn 50K đô la dựa trên các đặc điểm nhân trắc học của người đó.

2.1.4 Ứng dụng tiềm năng

Bộ dữ liệu có thể dùng để:

- Dự đoán thu nhập của một cá nhân trên hay dưới 50000 đô la/năm dựa trên các đặc điểm cá nhân của người đó.
- Phân tích mối quan hệ giữa thu nhập với các yếu tố như trình độ học vấn, loại hình công việc, thời gian làm việc, độ tuổi.
- Phân tích sự chênh lệch thu nhập giữa các nhóm dân cư, hỗ trợ ra quyết định trong kinh tế, xã hội.

2.2 Xử lý dữ liệu thiếu

2.2.1 Xác định loại Missing

Để xử lý giá trị thiếu, trước tiên chúng em sẽ xác định loại Missing của bộ dữ liệu. Chúng em tìm những trường nào bị thiếu bằng hàm `.info()`, và tính tỉ lệ phần trăm giá trị thiếu trong trường đó. Dưới đây là biểu đồ thể hiện phần trăm missing value ở các trường:

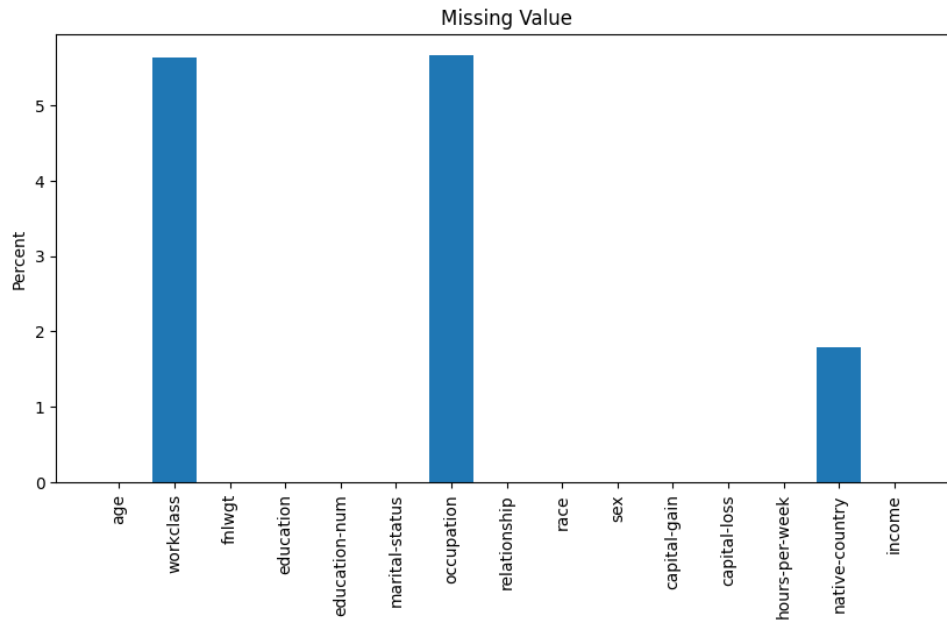


Figure 5: Missing data

Nhận xét: Các trường bị thiếu là **workclass**, **occupation**, **native-country** với tỉ lệ khá nhỏ.

Để xác định bộ dữ liệu thuộc loại nào cần xem xét sự phụ thuộc của 3 trường trên với các trường khác. Chúng em lựa chọn so sánh với biến mục tiêu là **income** và hai biến độc lập đại diện là **sex** và **education**.

Ảnh hưởng của giá trị thiếu đến cấu trúc của **income** được thể hiện trong các ảnh dưới đây:



Figure 6: Phần trăm missing data theo biến **income**

Nhận xét:

- **workclass:** nhóm bị thiếu workclass có tỷ lệ thu nhập >50K (10%) nhỏ hơn đáng kể so với nhóm không thiếu (25%), điều này cho thấy missing không xảy ra ngẫu nhiên.
- **occupation:** tương tự như workclass, nhóm bị thiếu occupation có tỷ lệ thu nhập >50K (10%) nhỏ hơn đáng kể so với nhóm không thiếu 25%.

- **native-country:** không có sự khác biệt nhiều ở nhóm bị thiếu và không bị thiếu (24% và 25%).

Ảnh hưởng của giá trị thiếu đến cấu trúc của **sex** được thể hiện trong các ảnh dưới đây:

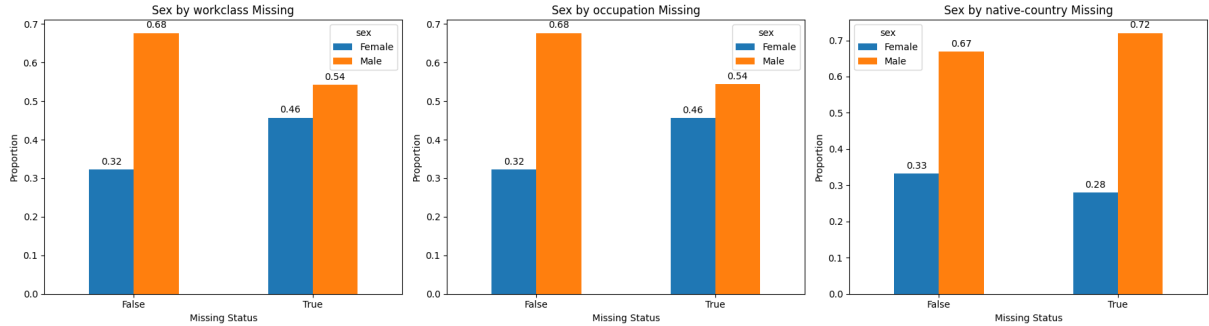


Figure 7: Phần trăm missing data theo biến **sex**

Nhận xét: Các quan sát bị thiếu ở các biến workclass, occupation và native-country có cơ cấu giới tính khác biệt so với nhóm dữ liệu đầy đủ. Đối với workclass và occupation, tỷ lệ nữ trong nhóm bị thiếu cao hơn đáng kể (46% với nhóm thiếu và 32% với nhóm đủ), trong khi với native-country, nhóm dữ liệu đủ lại có tỷ lệ nữ cao hơn (28% với nhóm thiếu và 33% với nhóm đủ). Điều này cho thấy dữ liệu bị thiếu không xuất hiện hoàn toàn ngẫu nhiên, mà có mối liên hệ với biến giới tính.

Chúng em tiến hành kiểm định **Chi-square** để kết luận có cơ sở hơn. Các thuộc tính được kiểm định ở đây là 3 thuộc tính thiếu với các thuộc tính categorical, bao gồm **education**, **marital-status**, **relationship**, **race**, **sex**.

Với các giả thuyết:

- H_0 : Hai biến độc lập.
- H_1 : Hai biến phụ thuộc.

Nếu $p\text{-value} < 0.05$ thì bác bỏ H_0 , nghĩa là hai biến phụ thuộc nhau. nghĩa là hai biến phụ thuộc nhau, ngược lại thì không đủ bằng chứng để bác bỏ H_0 . Chúng em chọn $\alpha = 0.05$ vì:

- $\alpha < 0.05$: dễ kết luận có phụ thuộc hơn dẫn đến nhiều false positive hơn.
- $\alpha > 0.05$: khó kết luận hơn nên ít false positive, nhưng dễ bỏ sót hiệu ứng thật (tăng lỗi loại II / false negative).

Do đó, 0.05 nằm giữa, không quá dễ, cũng không quá khắt khe. Sau khi chạy kiểm định thì chúng em thu được kết quả sau:

- **workclass:**

- **education:** $p\text{-value} = 1.4346740922569368 \times 10^{-54} < 0.05$
Có sự phụ thuộc giữa missing và education

- **marital-status:** $p_value = 4.045449521551414 \times 10^{-62} < 0.05$
Có sự phụ thuộc giữa missing và marital-status
- **relationship:** $p_value = 7.397950998921355 \times 10^{-75} < 0.05$
Có sự phụ thuộc giữa missing và relationship
- **race:** $p_value = 0.0008918600664578588 < 0.05$
Có sự phụ thuộc giữa missing và race
- **sex:** $p_value = 3.729618994301208 \times 10^{-32} < 0.05$
Có sự phụ thuộc giữa missing và sex

• **occupation:**

- **education:** $p_value = 4.146568231508675 \times 10^{-56} < 0.05$
Có sự phụ thuộc giữa missing và education
- **marital-status:** $p_value = 2.820650365542488 \times 10^{-62} < 0.05$
Có sự phụ thuộc giữa missing và marital-status
- **relationship:** $p_value = 8.381719622804803 \times 10^{-77} < 0.05$
Có sự phụ thuộc giữa missing và relationship
- **race:** $p_value = 0.0007323030495397479 < 0.05$
Có sự phụ thuộc giữa missing và race
- **sex:** $p_value = 5.798471160835414 \times 10^{-32} < 0.05$
Có sự phụ thuộc giữa missing và sex

• **native-country:**

- **education:** $p_value = 1.4584402625993669 \times 10^{-22} < 0.05$
Có sự phụ thuộc giữa missing và education
- **marital-status:** $p_value = 6.490613028482875 \times 10^{-7} < 0.05$
Có sự phụ thuộc giữa missing và marital-status
- **relationship:** $p_value = 0.0030687065259776382 < 0.05$
Có sự phụ thuộc giữa missing và relationship
- **race:** $p_value = 7.888171888947067 \times 10^{-69} < 0.05$
Có sự phụ thuộc giữa missing và race
- **sex:** $p_value = 0.009126820641745763 < 0.05$
Có sự phụ thuộc giữa missing và sex

Kết luận:

- Dựa vào biểu đồ cho thấy có sự khác biệt khá đáng kể ở nhóm bị thiếu và nhóm đủ ở cả 3 thuộc tính thiếu. Điều này cho thấy xác suất dữ liệu bị thiếu có mối liên hệ với các thuộc tính khác.
- Qua biểu đồ và kiểm định, có thể thấy cơ chế missing không hoàn toàn là ngẫu nhiên, có thể thuộc dạng MAR.
- Theo như em tìm hiểu thì để xác định bộ dữ liệu có thuộc MNAR thì phải dựa vào quy tắc nghiệp vụ hoặc thông tin ngoài dữ liệu quan sát được, do đó không thể kết luận MNAR dựa trên phân tích dữ liệu hiện có.

2.2.2 Điền dữ liệu

Do dữ liệu không có tính thời gian nên forward/backward fill không phù hợp. Vì cả ba thuộc tính thiếu đều thuộc loại dữ liệu categorical nên mean, median và K-NN không phù hợp, vì vậy mode imputation được áp dụng khá tốt cho các biến này. Ngoài ra, ta có thể encoding categorical trước rồi áp dụng K-NN imputation để so sánh, đánh giá. Nhưng nếu encoding được thì cả mean và median đều có thể áp dụng được, tuy nhiên, nếu dùng mean trong trường hợp này, giá trị có thể bị sai do tính toán ra số thập phân, đối với median, nếu chọn được trung vị thì cũng bất hợp lý do giá trị trung vị là do ta tự quy ước, áp đặt cấu trúc giả, dữ liệu bị bóp méo. Do đó, trong trường hợp này, chúng em sử dụng 2 imputation là mode và K-NN.

- **Mode Imputation:** Sử dụng SimpleImputer của thư viện Scikit-Learn với chiến lược **most_frequent** để điền dữ liệu còn thiếu.

Các biểu đồ sau đây thể hiện trước và sau khi imputation của các biến:

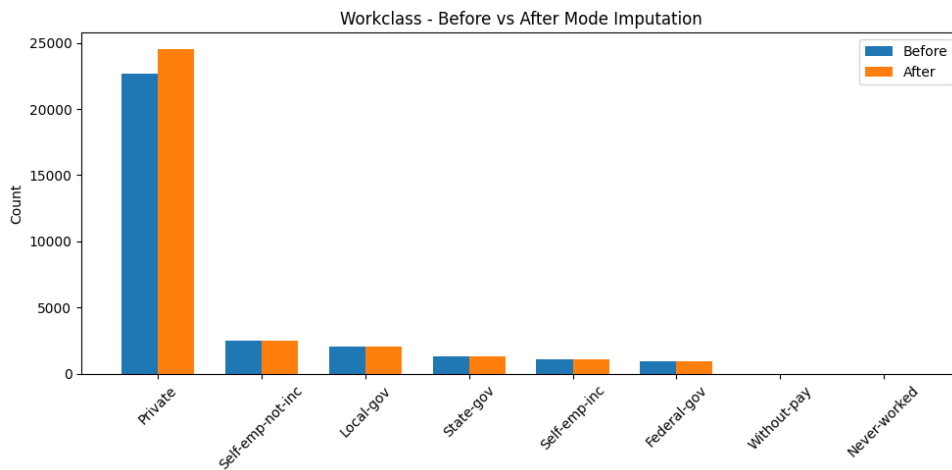


Figure 8: Workclass

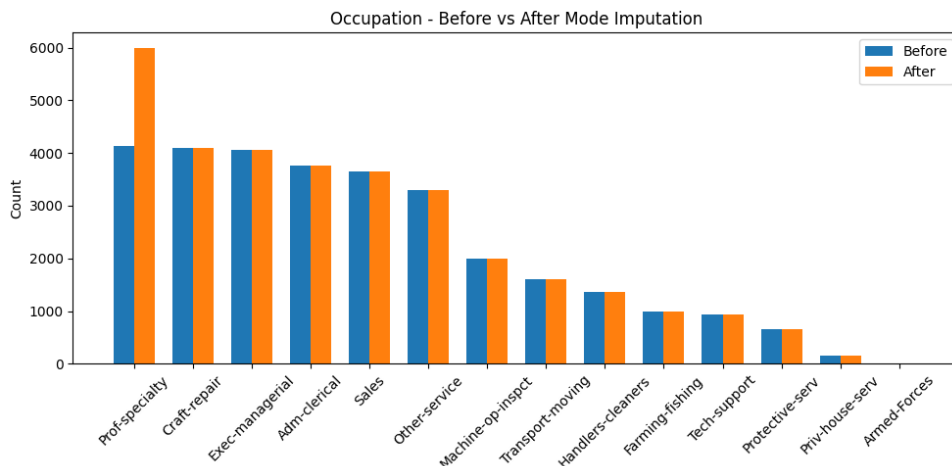


Figure 9: Occupation

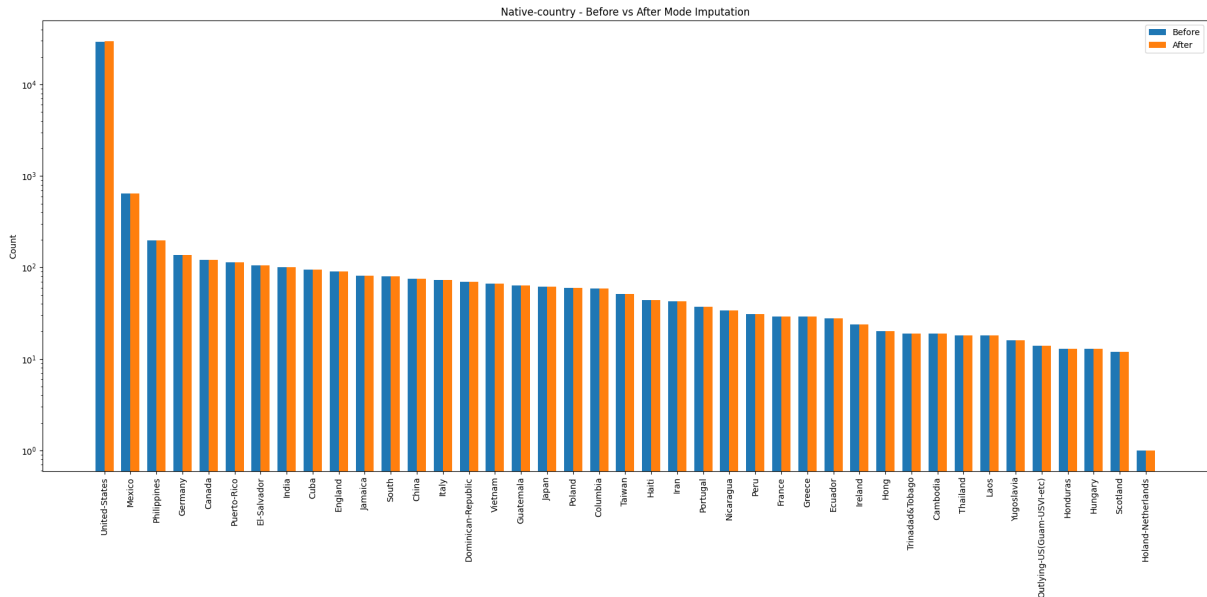


Figure 10: Native-country

- Ưu điểm: Phù hợp cho biến categorical như workclass, occupation, native-country. Dễ dàng thực hiện và tính toán, hiệu quả khi tỉ lệ thiếu nhỏ như bộ dữ liệu đang thực hiện (Các trường có tỉ lệ thiếu nhỏ <6%).
 - Nhược điểm: Làm thay đổi tần suất các biến nếu tỉ lệ thiếu lớn. Làm giảm phương sai, giảm sự đa dạng của dữ liệu, các mô hình học máy khó học được mối quan hệ giữa các dữ liệu.
- **K-NN Imputation:** Do thuật toán K-NN chỉ hoạt động trên dữ liệu dạng numeric, các biến categorical được chuyển đổi sang dạng biến giả (one-hot encoding) bằng hàm `get_dummies()`. Tham số `dummy_na=True` được sử dụng để biểu diễn giá trị thiếu của các biến phân loại như một nhãn riêng biệt.
 - Ưu điểm: Dữ liệu được điền có cơ sở, không máy móc và giữ được cấu trúc của dữ liệu.
 - Nhược điểm: Tốn nhiều chi phí, KNN phải lưu toàn bộ dữ liệu huấn luyện và tính toán mỗi khi dự đoán, gây tiêu tốn tài nguyên và thời gian khi xử lý khối lượng dữ liệu lớn. Cần phải one-hot nếu dữ liệu dạng categorical. Số chiều tăng mạnh.

Kết luận:

- Về chất lượng dữ liệu, mode và KNN đều giúp điền toàn bộ giá trị bị thiếu, nhưng mode hầu như giữ nguyên cấu trúc ban đầu của dữ liệu, trong khi KNN khai thác tốt mối quan hệ giữa các dữ liệu tuy nhiên làm tăng chi phí tính toán.
- Về phân bố dữ liệu, mode làm thay đổi không đáng kể cấu trúc ban đầu do missing nhỏ, trong khi KNN làm mượt phân bố dữ liệu sau khi mã hóa.

2.3 Chuẩn hóa dữ liệu

2.3.1 Đánh giá dữ liệu

Sau khi điền dữ liệu thì đến bước chuẩn hóa dữ liệu, trước tiên cần xác định các trường dạng numeric để xác định outlier. Các cột số gồm các cột: "**age**", "**education-num**", "**capital-gain**", "**capital-loss**", "**hours-per-week**". Sau khi xác định xong, chúng em quan sát các đặc điểm dữ liệu của các biến trên. Từ bảng, ta có thể thấy các biến **age**, **education-num** và **hours-per-week** có phân bố tương đối ổn định với trung bình và trung vị gần nhau. Ngược lại, hai biến **capital-gain** và **capital-loss** có phân bố lệch phải rất mạnh, với phần lớn giá trị bằng 0 và chỉ một số ít trường hợp có giá trị lớn, dẫn đến sự tồn tại của nhiều outlier. Bên cạnh đó, 2 biến **capital-gain** và **capital-loss** có max lớn hơn rất nhiều so với min.

Để thấy được phân bố của các giá trị ở từng biến, chúng em vẽ biểu đồ boxplot cho từng biến:



Figure 11: age

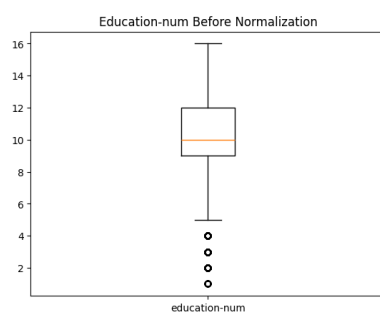


Figure 12: education-num

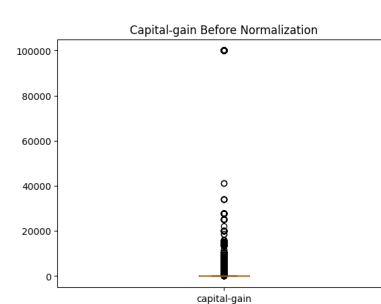


Figure 13: capital-gain

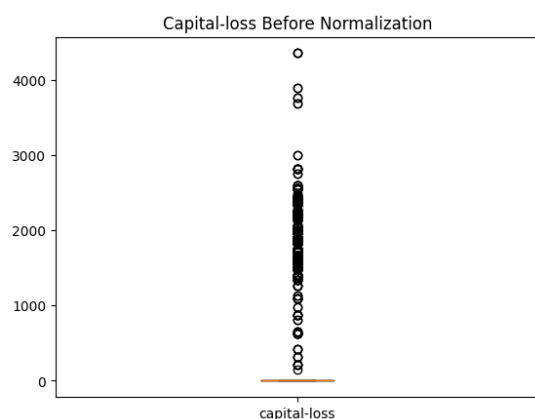


Figure 14: capital-loss

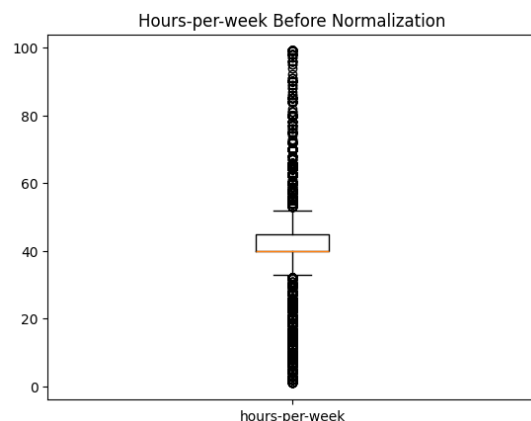


Figure 15: hour-per-week

Figure 16: Biểu đồ boxplot

Nhận xét: **capital-gain** và **capital-loss** phần lớn giá trị bằng 0, phân bố của cả hai lệch phải rất mạnh, xuất hiện rất nhiều outlier, các outlier này kéo trục y lên rất cao. Các biến **age**, **education-num** và **hour-per-week** xuất hiện một số giá trị ngoại lai với mật độ thưa, không lớn, **hour-per-week** phân bố tương đối

cân đối về 2 phía.

2.3.2 Chuẩn hóa

- **Min-Max Scaling:** Sử dụng `MinMaxScaler()` của class `sklearn.preprocessing`, đưa các giá trị về một khoảng cố định là $[0, 1]$ với công thức tổng quát:

$$X_{std} = \frac{X - min}{max - min}$$

$$X_{scaled} = X_{std} * (max - min) + min$$

Sau khi thực hiện Min-Max Scaling, ta thu được phân bố các biến của các giá trị như trong hình.

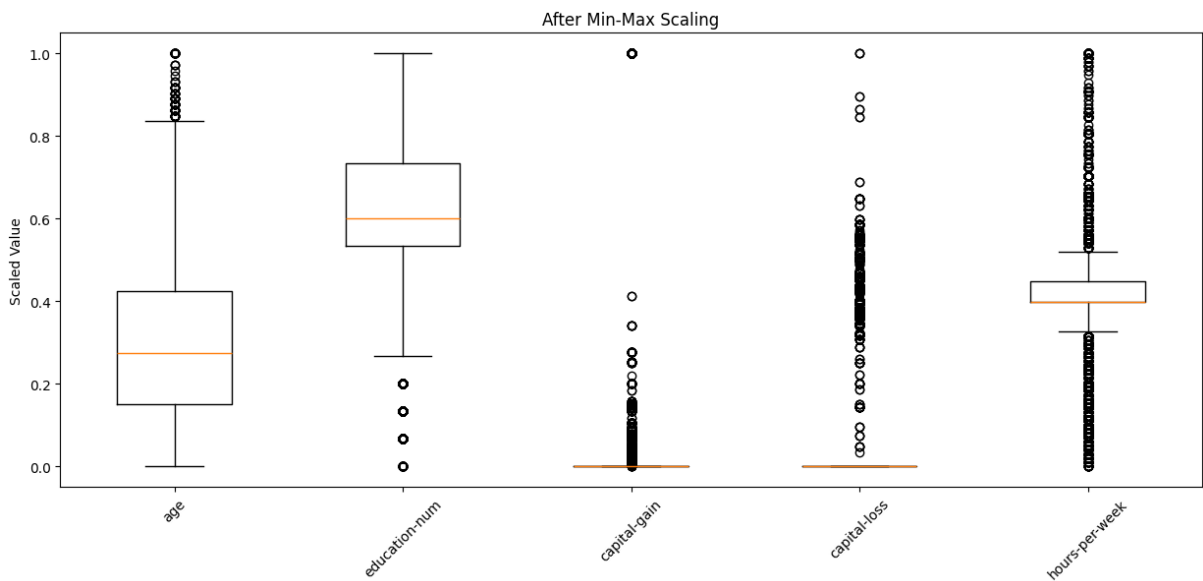


Figure 17: Phân bố giá trị sau khi Min-Max Scaling

- **Z-Score:** Sử dụng `StandardScaler()` của class `sklearn.preprocessing`, chuẩn hóa các đặc trưng bằng cách loại bỏ giá trị trung bình và đưa về phương sai bằng 1 với công thức tổng quát:

$$z = \frac{x - u}{s}$$

Sau khi thực hiện Standard Scaling, ta thu được phân bố các biến của các giá trị như trong hình.

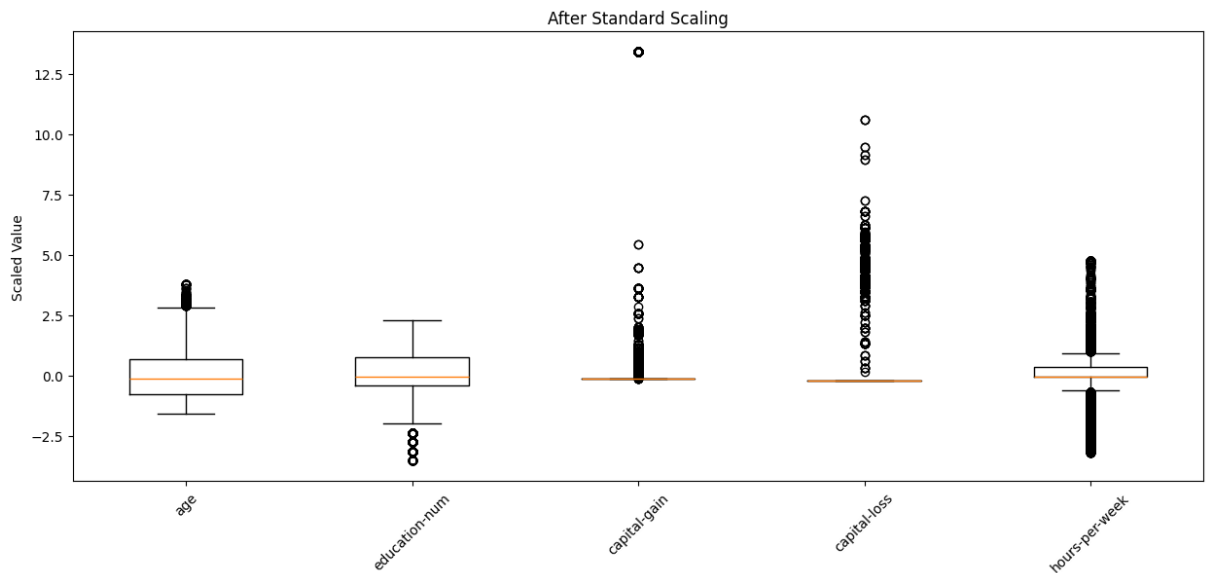


Figure 18: Phân bố giá trị sau khi Standard Scaling

Trong đó:

- **u**: giá trị trung bình
- **s**: độ lệch chuẩn
- **Robust Scaling**: Sử dụng `RobustScaler()` của class `sklearn.preprocessing`, Scaler này loại bỏ giá trị trung vị (median) và tỉ lệ hóa dữ liệu dựa trên khoảng tứ phân vị (mặc định là IQR - Interquartile Range).
IQR là khoảng nằm giữa tứ phân vị thứ nhất (25%) và tứ phân vị thứ ba (75%).

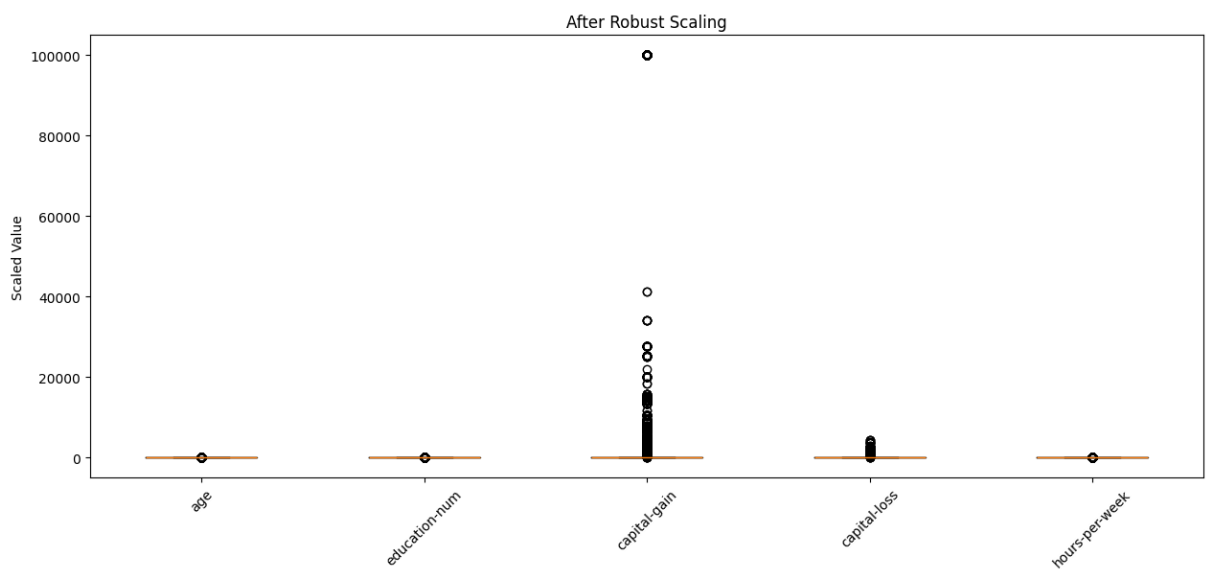


Figure 19: Phân bố giá trị sau khi Robust Scaling

So sánh:

- **Min - Max Scaling:**

- Giữ cấu trúc tốt.
- Bị ảnh hưởng mạnh bởi Outlier.
- Các biến lệch mạnh như capital-gain và capital-loss bị ép phẳng, làm mất nhiều thông tin.
- Không phù hợp với dataset hiện tại.

- **Z - Score:**

- Giữ cấu trúc tốt.
- Không loại bỏ được outlier mà còn làm dễ thấy outlier hơn.
- Không phù hợp với capital-gain và capital-loss vì giả định dữ liệu gần phân phối chuẩn, làm xuất hiện các giá trị cực lớn so với các biến còn lại (>10).
- Không phù hợp với dataset hiện tại.

- **Robust Scaling:**

- Giữ cấu trúc dữ liệu tốt, không làm méo dữ liệu trung tâm.
- Ít bị ảnh hưởng bởi outlier, chống outlier tốt nhất trong 3 phương pháp.
- Thang đo có thể không trực quan, giá trị có thể rất lớn.

Kết luận: Min-Max kém hiệu quả với biến lệch và nhiều outlier. Z - Score kém hiệu quả với outlier lớn, phù hợp với dữ liệu ít lệch hơn. Robust Scaling cho kết quả tốt nhất với dataset hiện tại do dữ liệu chứa nhiều biến có phân phối lệch mạnh và nhiều outlier như capital-gain và capital-loss.

2.4 Mã hóa biến phân loại

2.4.1 Xác định biến phân loại cần mã hóa

Vì các biến **Nominal** không có thứ tự, các biến **Ordinal** có phân cấp thứ bậc nên ta có thể xác định được dựa trên hiểu biết cá nhân, các biến thuộc 2 loại trên như sau:

- **Nominal:** workclass, marital-status, occupation, relationship, race, sex, native-country, income.
- **Ordinal:** education

Nhưng vì **education** đã có trường ordinal encoding sẵn là **education-num** nên ta sẽ không làm trên biến này và sẽ loại bỏ luôn biến này để tiện cho việc thực hiện câu d).

2.4.2 Mã hóa

One-hot encoding cho biến **Nominal** bằng hàm `OneHotEncoder()`. Vì `OneHotEncoder` không biết cột nào cần one-hot nên ta cần thêm `ColumnTransformer()` để phân công cột nào cần one-hot, cột nào cần ordinal, nhưng do biến ordinal là **education** đã one-hot rồi nên ta chỉ cần chỉ định cột cần one-hot.

Sau khi one-hot, chúng em kiểm tra lại chiều của bộ dữ liệu và tên các biến thì chiều tăng lên rất nhiều (từ 15 tăng lên 91).

2.4.3 Xử lý các thuộc tính phân loại có số lượng phần tử cao

Một số thuộc tính có nhiều loại phần tử (như **native-country** - 42 loại) sẽ làm tăng số chiều lên rất nhiều nhưng hầu hết các giá trị đều bằng 0. Bên cạnh đó, một số quốc gia rất ít xuất hiện làm cho bộ dữ liệu bị overfitting. Những điều này làm mô hình học sẽ khó hơn, tăng chi phí tính toán. Vì vậy cần có một số chiến lược phù hợp để giảm số chiều về một mức độ chấp nhận được. Dưới đây là một số chiến lược chúng em tìm hiểu được:

- **Nhóm các phân loại hiếm:** các phân loại có tần suất thấp (nhỏ hơn hoặc bằng một ngưỡng) sẽ gom chung lại với nhau thành cùng một phân loại là **other**. Điều này giúp làm giảm số chiều, giảm nhiễu mà còn giúp giữ được thông tin chính, không làm mất mát thông tin. Hơn nữa, nếu sau đó chúng ta có **Feature Selection** bằng **Variance Threshold** thì các cột hiếm gom chung sẽ tránh bị lược bỏ mất nếu tần suất lớn hơn ngưỡng.
- Mã hóa thành tần suất: nghĩa là chúng ta sẽ thay sự xuất hiện của một giá trị bằng tần suất xuất hiện của nó. Về biến **native-country**, ta có thể ví dụ thay giá trị United-States thành tần suất xuất hiện là 0.91 chẳng hạn.

2.5 Lựa chọn đặc trưng

- **Variance Threshold:** dùng `VarianceThreshold()` giúp loại bỏ tất cả các đặc trưng có phương sai thấp.

Thuật toán chọn đặc trưng này chỉ xem xét các đặc trưng (X), chứ không phải các kết quả mong muốn (y), và do đó có thể được sử dụng cho học không giám sát.

Trong bài làm của chúng em, sau khi encoding, số chiều tăng lên 91 nhưng sau khi áp dụng Variance Threshold, số chiều đã giảm khoảng phân nửa, số chiều chỉ còn 45. Tuy nhiên, dữ liệu giảm mạnh là do có nhiều giá trị hiếm của biến **native-country** và dữ liệu bị mất cân bằng. Thực chất dùng phương pháp này không được tốt cho bộ dữ liệu đang thực hiện, do bộ dữ liệu chứa biến categorical nhiều, sau khi encoding thì dữ liệu chủ yếu là 0 và 1, thường vẫn có variance khá lớn, rất ít cột bị loại. Phương pháp này chỉ hợp khi có cột gần như hằng số hoặc dữ liệu numeric thuần.

- **Apply feature importance from tree-based models:** phương pháp này xác định độ quan trọng của đặc trưng trong tập dữ liệu bằng thuật toán **Random Forest**.

Random forest là tập hợp của các cây quyết định **Decision Trees**.

Đối với bất kỳ cây quyết định nào, các đặc trưng được chọn bằng cách chúng phân vùng tập huấn thành các tập con thuần túy (cross entropy nhỏ nhất). Sau đó, một cây quyết định sẽ tạo một danh sách các bước để phân vùng dữ liệu hiệu quả nhất và theo số bước nhanh nhất. Vì vậy, các đặc trưng cao nhất trong cây (gần node gốc nhất) có thể được coi là quan trọng nhất.

Ta thực hiện huấn luyện **Random forest**, sử dụng thư viện **Sklearn**. Object **RandomForestClassifier** của **Sklearn** có thuộc tính **feature_importances_** là một list độ quan trọng của từng đặc trưng.

Trong **RandomForestClassifier** chúng em chọn **n_estimators=200** vì khi ít cây (10-50), mô hình dễ dao động, kết quả không ổn định, còn khi quá nhiều cây (>1000), cải thiện rất ít so với 100-300 cây nhưng tốn thời gian và chi phí. Vì vậy, 200 là mức cân bằng tốt giữa độ chính xác và tốc độ.

Tiếp theo, chúng em dùng **SelectFromModel()** để lựa chọn các đặc trưng có độ quan trọng lớn hơn hoặc bằng giá trị trung vị (median) của toàn bộ tập đặc trưng. Việc sử dụng ngưỡng median giúp loại bỏ khoảng một nửa số đặc trưng ít quan trọng nhất, từ đó giảm số chiều dữ liệu nhưng vẫn giữ lại các đặc trưng có ảnh hưởng lớn đến mô hình.

Sau đó, dữ liệu được biến đổi thông qua hàm **transform()** để giữ lại các đặc trưng được chọn.

Kết luận: Từ các kết quả, ban đầu có 91 features, **Variance Threshold** rút gọn còn 45 features và **Tree-based** đã rút gọn còn 46 features. Mặc dù số lượng đặc trưng còn lại gần tương đương, nhưng **Variance Threshold** loại bỏ dựa trên thống kê thuần túy, **Tree-based** loại bỏ dựa trên khả năng dự đoán. Do đó, **Tree-based selection** có ý nghĩa thực tế hơn trong **machine learning**.

Đối với Adult dataset, **Tree-based feature selection** là lựa chọn phù hợp hơn vì:

- Phù hợp với dữ liệu nhiều chiều sau khi one-hot trong khi **Variance Threshold** có thể loại bỏ các giá trị hiếm mặc dù chúng vẫn có thể mang giá trị phân biệt
- Đánh giá đặc trưng dựa trên khả năng phân loại.
- Phát hiện mối quan hệ giữa các biến.
- Ít bị ảnh hưởng bởi phân phối dữ liệu. Trong khi **Variance Threshold** phụ thuộc trực tiếp vào phương sai.

Vì vậy, trong bài toán phân loại thu nhập (>50K), **Tree-based selection** được ưu tiên hơn **Variance Threshold**.

3 Tiền xử lý dữ liệu văn bản

3.1 Mô tả bộ dữ liệu

Tên bộ dữ liệu: IMDB Movie Reviews

Nguồn: [IMDB Dataset of 50K Movie Reviews](#)

Quy mô và cấu trúc: Tập dữ liệu gồm 50.000 bài đánh giá phim, được chia cân bằng thành 25.000 mẫu tích cực (*positive*) và 25.000 mẫu tiêu cực (*negative*). Mỗi mẫu gồm hai thuộc tính:

- **review:** văn bản tự do có độ dài biến thiên lớn.
- **sentiment:** nhãn nhị phân.

Đặc điểm dữ liệu:

- Văn bản không cấu trúc.
- Chứa HTML tags và ký tự đặc biệt.
- Độ dài review không đồng nhất.
- Nhiều biểu đạt cảm xúc mạnh và từ lóng.

Use case:

- Binary sentiment classification.
- Benchmark cho NLP truyền thống và Deep Learning.

3.2 Tokenization

3.2.1 Lý do sử dụng

Văn bản IMDB ở dạng thô, không có ranh giới rõ ràng giữa các đơn vị ngôn ngữ. Mục tiêu của bước này là chia văn bản thành các đơn vị nhỏ hơn (tokens) như từ, câu hoặc subwords. Để tạo input tiêu chuẩn cho các bước xử lý tiếp theo Tokenization là bước bắt buộc để:

- Chuyển văn bản thành đơn vị xử lý (tokens).
- Chuẩn bị đầu vào cho vectorization.
- Giảm nhiễu từ ký tự đặc biệt và HTML.

Trong phần này em sử dụng 3 kỹ thuật khác nhau:

- Word Tokenization (NLTK, spaCy) [5]: Tách văn bản thành các từ riêng lẻ
- Sentence Tokenization (NLTK, spaCy): Tách đoạn văn thành các câu độc lập
- Subword Tokenization (BERT – WordPiece): Xử lý các từ phức tạp và từ hiếm

3.2.2 Triển khai

Word tokenization được triển khai bằng hai thư viện xử lý ngôn ngữ tự nhiên phổ biến: **NLTK (Natural Language Toolkit)** và **spaCy**.

Với NLTK, quá trình tách từ được thực hiện thông qua hàm `word_tokenize()` thuộc module `nltk.tokenize`. Hàm này sử dụng tokenizer dựa trên quy tắc để phân tách văn bản thành các token ở mức từ.

Với spaCy, tokenization được thực hiện bằng cách truyền văn bản vào pipeline NLP thông qua lệnh: `doc_spacy = nlp(sample_review)`. Đối tượng `Doc` trả về chứa các token được tạo bởi tokenizer tích hợp của spaCy, sau đó các token được trích xuất thông qua thuộc tính `token.text`.

Khác với NLTK, `nlp()` trong spaCy kích hoạt toàn bộ pipeline xử lý (bao gồm tokenization, POS tagging, dependency parsing nếu được bật), do đó chi phí tính toán cao hơn nhưng cho phép mở rộng sang các tác vụ phân tích ngôn ngữ sâu hơn.

Lý do chọn NLTK thay vì spaCy:

- spaCy kích hoạt full NLP pipeline (POS, dependency parsing).
- Chi phí tính toán cao hơn đáng kể.
- Không cần thiết cho bài toán TF-IDF + Logistic Regression.

Computational considerations:

Độ phức tạp xấp xỉ $O(n)$ theo độ dài văn bản. Với 50.000 reviews, NLTK đảm bảo thời gian xử lý chấp nhận được mà không làm tăng memory overhead.

3.2.3 Kết quả

Thực nghiệm trên 1.000 reviews cho thấy:

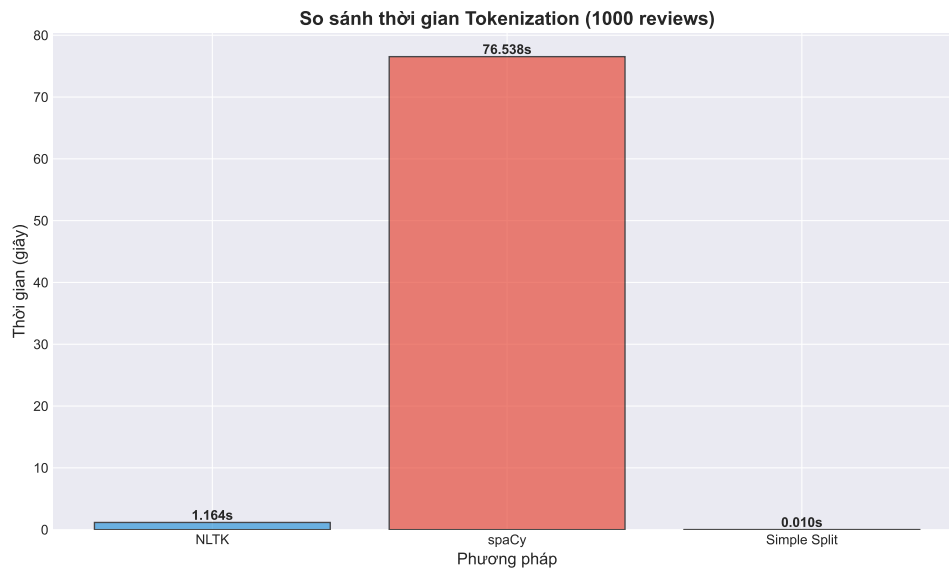


Figure 20: So sánh tốc độ giữa NLTK và spaCy

- Simple split nhanh nhất nhưng kém chính xác.
- NLTK cân bằng tốt giữa tốc độ và chất lượng.
- spaCy chậm hơn đáng kể.

Do đó, NLTK được chọn để đảm bảo hiệu quả tính toán và tính nhất quán pipeline.

3.3 Stop-word Removal

3.3.1 Lý do sử dụng

Stop words xuất hiện với tần suất cao nhưng mang ít giá trị phân biệt. Loại bỏ chúng giúp:

- Giảm dimensionality.
- Giảm sparsity trong ma trận đặc trưng.
- Giảm chi phí tính toán.

Tuy nhiên, trong sentiment analysis, một số stop words như *not*, *no*, *never* mang ý nghĩa quan trọng.

3.3.2 Triển khai

Stop words được lấy từ:

```
stopwords.words('english')
```

Việc loại bỏ được thực hiện bằng hàm:

```
remove_stopwords(tokens, stop_words)
```

với điều kiện:

- Chỉ giữ token alphabetic (`token.isalpha()`)
- Loại bỏ nếu xuất hiện trong danh sách stop words

Sau đó áp dụng toàn bộ dataset:

```
df['tokens_no_stopwords'] = df['tokens'].apply(...)
```

Justification:

Giữ lại từ phủ định nhằm tránh đảo ngược polarity (ví dụ: “not good”).

Computational considerations:

Loại bỏ stop words giúp giảm khoảng 40% số lượng tokens, từ đó:

- Giảm kích thước vocabulary.
- Giảm memory footprint.
- Tăng tốc bước vectorization.

3.3.3 Kết quả

Kết quả cho thấy:

- Tokens giảm 43.9%.
- Vocabulary giảm 30%.
- Accuracy giảm 0.8%.



Figure 21: Phân phối trước/sau khi loại bỏ stop words

Phân phối cho thấy sau khi loại stop words, các từ mang nội dung như “movie”, “film”, “one” trở nên nổi bật hơn.

Table 1: So sánh hiệu năng phân loại (Có và Không loại bỏ Stopwords)

Chỉ số	Có Stopwords	Không Stopwords
Độ chính xác (Accuracy)	0.9065	0.9135
Độ chính xác dương (Precision)	0.9018	0.9078
Độ bao phủ (Recall)	0.9134	0.9214
F1-Score	0.9076	0.9146
Kích thước từ vựng	1000	1000

Kết quả cho thấy việc loại bỏ stopwords giúp cải thiện đồng đều tất cả các chỉ số đánh giá (Accuracy, Precision, Recall, F1-Score). Tuy nhiên, phân tích ngữ nghĩa cho thấy một số từ phủ định quan trọng như “not” và “no” xuất hiện với tần suất cao, do đó cần cân nhắc giữ lại các từ này để tránh làm sai lệch polarity trong sentiment classification.

3.4 Stemming và Lemmatization

3.4.1 Lý do sử dụng

Chuẩn hóa giúp [6]:

- Giảm sparsity.
- Gom các biến thể về cùng một đặc trưng.
- Cải thiện tính ổn định của mô hình.

3.4.2 Triển khai

Ba phương pháp được áp dụng trên toàn bộ dataset:

- Porter Stemmer: `PorterStemmer()`
- Snowball Stemmer: `SnowballStemmer('english')`
- Lemmatization: `WordNetLemmatizer()` kết hợp POS tagging

Lemmatization sử dụng hàm ánh xạ POS:

`nltk.pos_tag() + get_wordnet_pos()`

Toàn bộ 50.000 reviews được xử lý và đo:

- Thời gian chạy từng phương pháp
- Vocabulary size sau mỗi phương pháp

- Tỷ lệ giảm vocabulary

So sánh tham số và lựa chọn:

- Stemming: nhanh, không dùng từ điển.
- Lemmatization: chậm hơn, nhưng bảo toàn ngữ nghĩa.

Computational considerations:

Lemmatization yêu cầu tra cứu từ điển và POS tagging sẽ tăng chi phí xử lý, nhưng chấp nhận được với 50.000 mẫu.

3.4.3 Kết quả

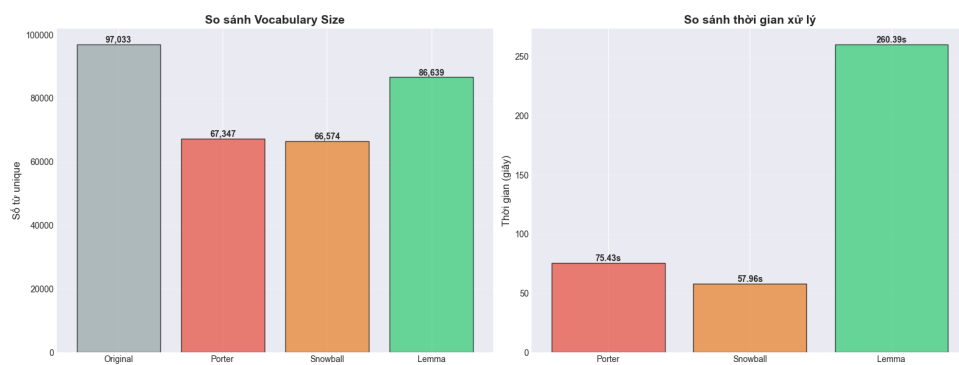


Figure 22: So sánh Vocabulary Size sau Stemming và Lemmatization

- Stemming giảm vocabulary 34–35%.
- Lemmatization giảm 25%.
- Lemmatization bảo toàn semantic tốt hơn.

Vì mục tiêu là sentiment classification với khả năng diễn giải feature, lemmatization được lựa chọn.

3.5 Text Vectorization

3.5.1 Lý do sử dụng

Văn bản cần được chuyển sang dạng số để mô hình học máy xử lý. Vectorization quyết định trực tiếp đến hiệu năng mô hình.

3.5.2 Cơ sở toán học

Giả sử tập dữ liệu gồm N documents và vocabulary gồm V từ:

$$X \in \mathbb{R}^{N \times V}$$

Trong đó:

- N : số documents
- V : số đặc trưng (features)
- X_{ij} : trọng số của từ j trong document i

3.5.3 Bag of Words (BoW)

BoW biểu diễn document bằng tần suất xuất hiện của từ:

$$X_{ij} = tf_{ij}$$

Trong đó tf_{ij} là số lần từ j xuất hiện trong document i .

Ưu điểm:

- Đơn giản, dễ triển khai
- Dễ diễn giải đặc trưng

Nhược điểm:

- Không xét đến độ phổ biến toàn cục của từ
- Dễ bị ảnh hưởng bởi các từ xuất hiện thường xuyên

3.5.4 TF-IDF

TF-IDF cải tiến BoW bằng cách điều chỉnh trọng số dựa trên độ hiếm của từ trong toàn bộ tập dữ liệu.

$$TF_{ij} = \frac{f_{ij}}{\sum_k f_{ik}}$$

$$IDF_j = \log \left(\frac{N}{df_j} \right)$$

$$TFIDF_{ij} = TF_{ij} \times IDF_j$$

Trong đó:

- f_{ij} : số lần từ j xuất hiện trong document i
- df_j : số documents chứa từ j
- N : tổng số documents

Trong triển khai thực tế, tham số `sublinear_tf=True` sử dụng log-scaling:

$$TF_{ij} = 1 + \log(f_{ij})$$

Giúp giảm ảnh hưởng của các từ có tần suất quá cao trong một document.

3.5.5 Word2Vec

Word2Vec học embedding dense thông qua tối ưu hóa xác suất xuất hiện của từ trong ngữ cảnh.

Với kiến trúc CBOW:

$$\max \prod_{t=1}^T P(w_t | \text{context}(w_t))$$

Vector mỗi từ có kích thước $d = 100$, và vector của document được tính bằng trung bình các word embeddings:

$$\vec{d}_i = \frac{1}{|W_i|} \sum_{w \in W_i} \vec{w}$$

Trong đó:

- W_i : tập từ trong document i
- $\vec{w}$: vector embedding của từ w

3.5.6 Triển khai

Sau khi hoàn tất tiền xử lý, các tokens được nối lại thành chuỗi văn bản thông qua:

```
df['processed_text'] = df['tokens_lemma'].apply(lambda x: ' '.join(x))
```

Ba phương pháp vectorization được triển khai gồm BoW, TF-IDF và Word2Vec.

1. Bag-of-Words (BoW)

Sử dụng `CountVectorizer` với các tham số:

- `max_features=5000`: giới hạn 5000 từ phổ biến nhất.
- `min_df=5`: loại bỏ từ xuất hiện dưới 5 documents.
- `max_df=0.8`: loại bỏ từ xuất hiện trên 80% documents.
- `ngram_range=(1,2)`: sử dụng unigram và bigram.

Ma trận đặc trưng được tạo bằng:

```
bow_matrix = bow_vectorizer.fit_transform(df['processed_text'])
```

Kết quả là ma trận sparse kích thước $(N \times 5000)$.

2. TF-IDF

Sử dụng `TfidfVectorizer` với cùng cấu hình như BoW và bổ sung:

- `sublinear_tf=True`: áp dụng log-scaling cho term frequency.

Ma trận TF-IDF được tạo bằng:

```
tfidf_matrix = tfidf_vectorizer.fit_transform(df['processed_text'])
```

TF-IDF giảm trọng số của các từ phổ biến và làm nổi bật từ đặc trưng.

3. Word2Vec

Embedding được huấn luyện bằng mô hình Word2Vec (CBOW) với:

- vector_size=100
- window=5
- min_count=5
- epochs=10

Vector của mỗi document được tính bằng trung bình các word embeddings:

```
doc_vector = np.mean([w2v_model.wv[w] for w in tokens], axis=0)
```

Kết quả là biểu diễn dense vector kích thước 100 chiều cho mỗi document.

Justification:

- Bigram giúp nắm bắt cụm phủ định như “not good”.
- Giới hạn số chiều giúp giảm chi phí tính toán.

Computational considerations:

Ma trận TF-IDF là sparse matrix (mật độ 1–3%). Thư viện sử dụng CSR format để tối ưu bộ nhớ.

Độ phức tạp xấp xỉ:

$$O(N \times V)$$

trong đó N là số document và V là vocabulary size.

3.5.7 Kết quả

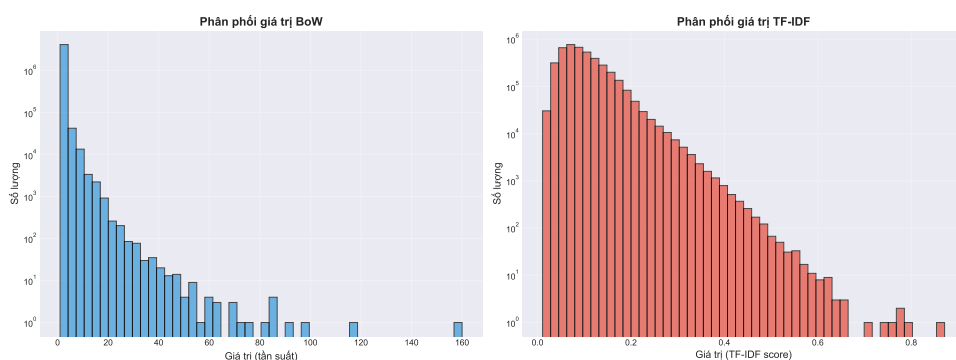


Figure 23: Phân phối của BoW và TF-IDF

Phân phối giá trị của BoW lệch mạnh về phía các tần suất thấp và tồn tại đuôi dài do một số từ xuất hiện rất thường xuyên. Điều này khiến mô hình dễ bị ảnh hưởng bởi các từ phổ biến nhưng ít giá trị phân biệt.

Ngược lại, TF-IDF tạo ra phân phối ổn định và phân tán hơn, với phần lớn trọng số nằm trong khoảng thấp. Cơ chế chuẩn hóa theo IDF giúp giảm ảnh hưởng của từ phổ biến và làm nổi bật các từ đặc trưng.

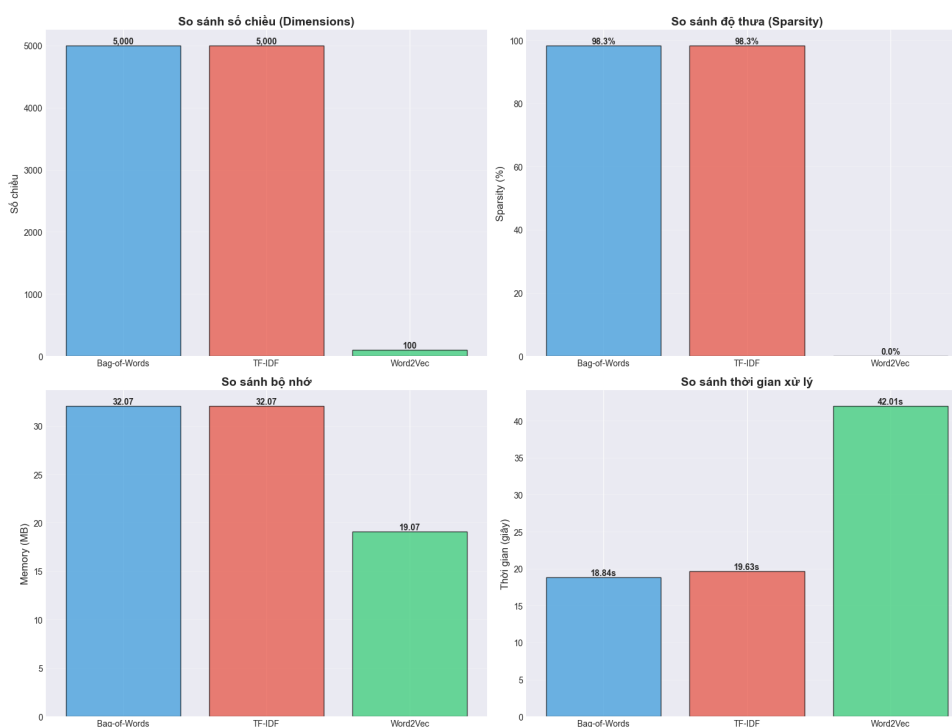


Figure 24: So sánh BoW và TF-IDF

BoW và TF-IDF có số chiều lớn (5,000) và độ thưa rất cao (98%), trong khi Word2Vec tạo vector dense 100 chiều.

Mặc dù Word2Vec tiết kiệm bộ nhớ hơn (19MB so với 32MB), thời gian xử lý cao hơn đáng kể do chi phí huấn luyện embedding.

BoW có tốc độ nhanh nhất, nhưng TF-IDF mang lại biểu diễn ổn định và có tính phân biệt cao hơn. Xét tổng thể giữa hiệu quả và chi phí tính toán, TF-IDF được lựa chọn làm biểu diễn chính cho bài toán phân loại cảm xúc.

Thực nghiệm cho thấy Logistic Regression với TF-IDF đạt độ chính xác cao hơn so với BoW, chứng tỏ TF-IDF cung cấp biểu diễn đặc trưng hiệu quả hơn cho bài toán phân loại cảm xúc.

4 Tiền xử lý dữ liệu chuỗi thời gian

4.1 Mô tả Dữ liệu

4.1.1 Temporal Data

Temporal data (dữ liệu theo thời gian) là dữ liệu mà mỗi quan sát gắn liền với một mốc thời gian cụ thể, và trình tự thời gian của dữ liệu là quan trọng.

Đặc điểm nổi bật:

- Có timestamp (ngày, giờ, phút, giây...).
- Có thứ tự.
- Thường có xu hướng hoặc chu kỳ.
- Các giá trị ở gần nhau thường có liên quan với nhau.

4.1.2 Dataset sử dụng

Dataset lựa chọn [Stock Market Data \(NASDAQ, NYSE, S&P500\)](#) (Một trong bốn Dataset được thầy gợi ý).

Nguồn gốc Dataset được đăng trên Kaggle với tác giả là Paul Mooney (không có thông tin citation gì thêm).

4.1.3 Quy mô dữ liệu

Dataset bao gồm dữ liệu lịch sử giá và khối lượng giao dịch của các cổ phiếu niêm yết trên các sàn giao dịch lớn của Hoa Kỳ. Kích thước dữ liệu:

- Số dòng: 10,592 dòng
- Số cột: 7 thuộc tính chính

4.1.4 Mô tả chi tiết

Dataset được tổ chức theo thư mục

a. Cấu trúc thư mục

```
data/temporal/stock_market_data/
|-- forbes2000/      (2246 files, 1076 files .csv)
|-- nasdaq/          (3617 files, 1564 files .csv)
|-- nyse/            (2516 files, 1145 files .csv)
'-- sp500/           (831 files)
    |-- csv/         (409 file CSV)
    '-- json/        (422 file JSON)
```

Tổng cộng: ~4194 files .csv (mỗi file tương ứng với 1 mã cổ phiếu)

b. Cấu trúc file CSV

Mỗi file CSV chứa dữ liệu lịch sử của một mã cổ phiếu với các cột:

Cột	Kiểu dữ liệu	Mô tả
Date	String (DD-MM-YYYY)	Ngày giao dịch
Open	Float	Giá mở cửa
High	Float	Giá cao nhất trong ngày
Low	Float	Giá thấp nhất trong ngày
Close	Float	Giá đóng cửa
Volume	Integer	Khối lượng giao dịch
Adjusted Close	Float	Giá đóng cửa đã điều chỉnh

4.1.5 Tiềm năng sử dụng

Dataset Stock Market Data có nhiều tiềm năng ứng dụng trong các bài toán phân tích và dự báo chuỗi thời gian:

- Hướng đầu tiên là cho những mục tiêu thực tế như: phân tích xu hướng, dự báo giá cổ phiếu, phát hiện biến động, xây dựng chiến lược đầu tư,... gọi chung là để khảo sát, nghiên cứu phục vụ cho Kinh tế, đầu tư.
- Hướng thứ hai là phục vụ cho học tập (như tại em). Do lượng dữ liệu dồi dào và chứa đựng nhiều ý nghĩa nên đây là một nguồn tài nguyên tuyệt vời cho học sinh, sinh viên, nhà nghiên cứu tìm tòi khám phá.

Do dữ liệu có tần suất liên tục (mỗi ngày) và thời gian kéo dài nhiều năm, dataset đặc biệt phù hợp cho các kỹ thuật xử lý temporal data (bao gồm cả những kỹ thuật được thầy yêu cầu khai thác).

4.1.6 Lựa chọn của chúng em

Do dữ liệu quá nhiều với mỗi file .csv là thông tin 1 mã cổ phiếu trong giai đoạn 1980-2020 (có 4194 files .csv). Sau quá trình khảo sát và được sự chấp nhận của thầy, chúng em quyết định chỉ chọn một mã cổ phiếu trên một sàn giao dịch (tương ứng với 1 file .csv) để xử lý.

Chúng em quyết định lựa chọn mã cổ phiếu Apple Inc. (AAPL) từ sàn S&P 500 để phân tích.

Lý do:

- AAPL có lịch sử giao dịch từ năm 1980 đến nay (khoảng 44 năm), thỏa yêu cầu về dữ liệu.
- Apple là một trong những cổ phiếu được giao dịch nhiều nhất trên thị trường. Vì vậy chúng em kì vọng sẽ khai thác được nhiều thông tin thú vị.

4.2 Kỹ thuật tiền xử lý áp dụng

4.2.1 Parsing Date and Time

a. Cơ sở và lý do thực hiện

Ban đầu cột Date tồn tại dưới dạng chuỗi ký tự. Ở kiểu dữ liệu này, hệ thống không thể hiểu được bản chất thứ tự thời gian, cũng như không thể thực hiện các thao tác chuyên biệt cho temporal data.

Việc parsing DateTime là quá trình chuyển đổi dữ liệu thời gian từ dạng chuỗi sang kiểu `datetime`. Đây là bước tiền xử lý nền tảng nhằm đảm bảo rằng dữ liệu được xử lý đúng theo trục thời gian thay vì theo thứ tự ngẫu nhiên.

Bước Parsing Date and Time là cần thiết vì các lý do sau:

- Cho phép hệ thống nhận diện đúng thứ tự thời gian và tránh sai lệch khi chia tập train-test.
- Hỗ trợ truy xuất các thành phần như year, month, day.
- Đảm bảo việc sắp xếp dữ liệu theo trình tự thời gian tăng dần trước khi phân tích, thực hiện các bước tiền xử lý phía sau..

Nếu không thực hiện bước này, toàn bộ quá trình phân tích chuỗi thời gian phía sau có thể bị sai lệch do dữ liệu không được tổ chức đúng cấu trúc thời gian.

b. Triển khai chi tiết

Quy trình thực hiện gồm các bước:

- Kiểm tra kiểu dữ liệu ban đầu của cột Date.
- Sử dụng `pd.to_datetime()` và chỉ định rõ định dạng `%d-%m-%Y` để tránh nhầm lẫn giữa các trường thông tin.
- Thiết lập Date làm index của DataFrame.
- Sắp xếp lại dữ liệu tăng dần theo index vừa đặt.
- Kiểm tra lại phạm vi thời gian để xác nhận việc chuyển đổi thành công.

Về mặt chi phí tính toán, quá trình chuyển đổi có độ phức tạp tuyến tính với số dòng dữ liệu. Với 10,592 dòng, thời gian xử lý là rất nhỏ và không ảnh hưởng đến hiệu năng tổng thể.

c. Kết quả và nhận xét



Figure 25: Dữ liệu giá Close của AAPL theo thời gian

Sau khi hoàn tất bước parsing:

- Cột Date đã được chuyển sang kiểu `datetime64[ns]`.
- Date được thiết lập làm index của DataFrame.
- Dữ liệu được sắp xếp đúng theo thứ tự thời gian tăng dần.
- Phạm vi dữ liệu trải dài từ năm 1980 đến 2020.

4.2.2 Handling Time Gaps

a. Cơ sở và lý do thực hiện

Dataset bao gồm dữ liệu giá cổ phiếu theo ngày giao dịch. Khi tính khoảng cách giữa các timestamp liên tiếp, có thể quan sát thấy tồn tại các khoảng cách lớn hơn 1 ngày. Các khoảng trống này chủ yếu xuất hiện do cuối tuần và ngày nghỉ lễ – tức là những ngày thị trường không giao dịch.

Tuy nhiên, đối với nhiều kỹ thuật xử lý chuỗi thời gian (đặc biệt là một số mô hình yêu cầu tần suất cố định), việc dữ liệu không liên tục theo thời gian có thể gây ảnh hưởng đến quá trình xây dựng feature như lag, rolling window hoặc resampling.

Vì vậy, chúng em tiến hành:

- Xác định và thống kê các khoảng cách thời gian lớn hơn 1 ngày.
- Reindex dữ liệu theo toàn bộ calendar day trong khoảng thời gian quan sát.
- So sánh các phương pháp điền giá trị để lựa chọn kỹ thuật phù hợp nhất.

Trong bối cảnh này, Handling Time Gaps không chỉ dừng lại ở việc phát hiện khoảng trống, mà còn bao gồm đánh giá tác động của việc chuẩn hóa lại trục thời gian.

b. Triển khai chi tiết

Quy trình thực hiện gồm các bước:

- Kiểm tra time gaps của dữ liệu.
- Tiến hành khảo sát và phân tích time gaps.
- Áp dụng cả 5 phương pháp fill dữ liệu mà đề yêu cầu cho một cột để khảo sát.
- Chọn ra phương pháp phù hợp, tiến hành fill cho toàn bộ dữ liệu.

Quy trình kiểm tra được thực hiện như sau:

- Tính hiệu giữa các timestamp liên tiếp.
- Xác định các khoảng cách lớn hơn 1 ngày.
- Đối chiếu các khoảng trống với lịch giao dịch để xác nhận đó là cuối tuần hoặc ngày nghỉ.

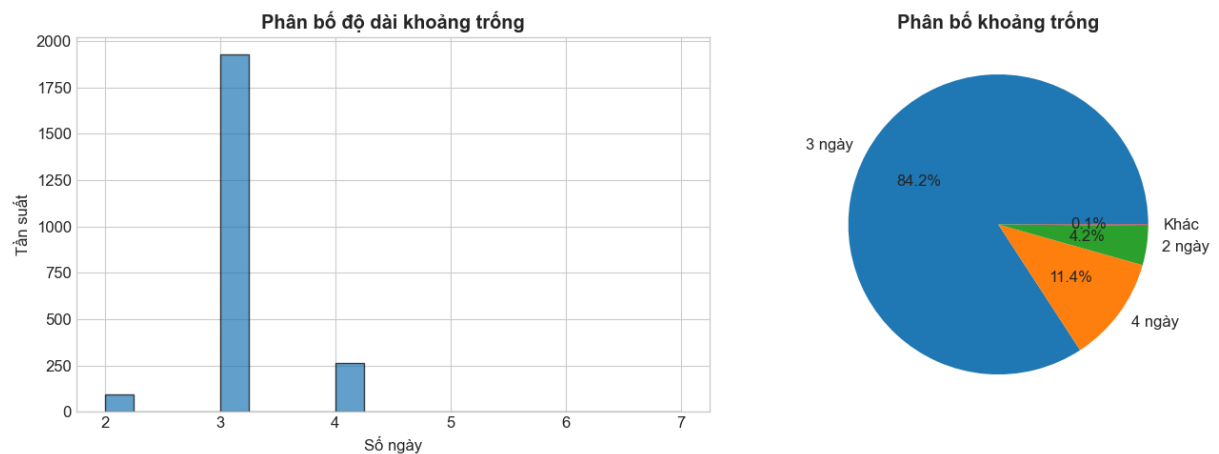


Figure 26: Thống kê khoảng trống trên Dataset

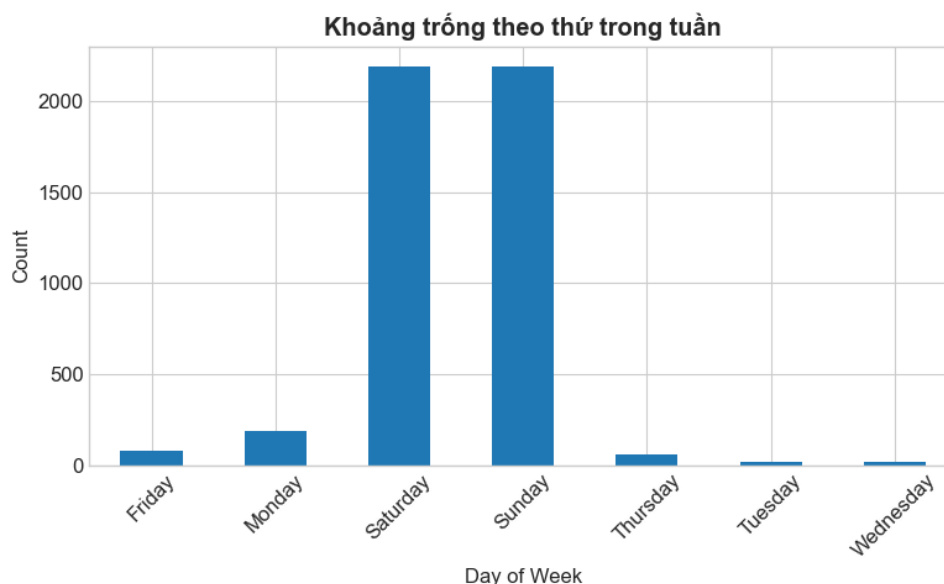


Figure 27: Thống kê khoảng trống theo thứ trong tuần

Kết quả cho thấy phần lớn các time gaps rơi vào thứ Bảy, Chủ Nhật.

Sau khi tìm hiểu dữ liệu stock, chúng em nhận thấy: không có dữ liệu vào ngày nghỉ và ngày lễ, nên đây là ngày không giao dịch chứ không phải dữ liệu bị thiếu.

Tuy nhiên, đối với nhiều kỹ thuật xử lý time-series, việc dữ liệu không liên tục có thể gây ảnh hưởng (trong bài làm này nếu không liên tục sẽ khó tạo lag features, differences features, rolling statistics). Nên chúng em chọn sẽ fill cho toàn bộ ngày.

Tiến hành thử nghiệm 5 phương pháp fill dữ liệu:

- Forward Fill
- Backward Fill
- Linear Interpolation
- Polynomial Interpolation (bậc 2)
- Spline Interpolation (bậc 3)

Kết quả cho thấy:

- Forward Fill và Backward Fill tạo ra dạng bậc thang do kế thừa giá trị lân cận.
- Linear, Polynomial và Spline tạo ra đường nối mượt giữa các điểm, nhưng các giá trị này mang tính nội suy, không phản ánh giá được hình thành từ giao dịch thực tế.
- Backward Fill sử dụng thông tin từ tương lai, tiềm ẩn nguy cơ data leakage nếu áp dụng trong mô hình dự báo.

c. Lựa chọn phương pháp và áp dụng

Sau khi phân tích, chúng em lựa chọn **Forward Fill** vì các lý do sau:

- Phù hợp với bản chất dữ liệu tài chính: trong ngày không giao dịch, giá đóng cửa được xem như giữ nguyên từ phiên gần nhất.
- Không sử dụng thông tin tương lai, do đó không gây data leakage.
- Không tạo ra giá trị nhân tạo như các phương pháp nội suy.

Forward Fill sau đó được áp dụng cho toàn bộ các cột giá (Open, High, Low, Close) và Volume trong dataset đã reindex.

d. Kết quả và nhận xét

Sau bước xử lý này, dữ liệu đạt được:

- Trục thời gian liên tục, không còn giá trị thiếu.
- Dữ liệu đã sẵn sàng cho các bước xử lý temporal tiếp theo.

4.2.3 Extracting Time-Based Features

a. Cơ sở và lý do thực hiện

Trong dữ liệu chuỗi thời gian tài chính, bản thân timestamp không chỉ mang ý nghĩa về thứ tự quan sát mà còn hàm chứa thông tin về seasonality, calendar effects và các pattern lặp lại theo năm, theo quý hoặc theo tuần.

Lý do cần thực hiện

- Giúp mô hình học được tính chu kỳ theo tháng, theo quý và theo năm.
- Phân tích hiệu ứng ngày trong tuần.
- Khai thác các calendar effects như đầu tháng, cuối quý, cuối năm.
- Biểu diễn các đặc trưng mang tính chu kỳ để tránh vấn đề gián đoạn tuần hoàn.

Những hạn chế và rủi ro có thể phát sinh Nếu bổ sung quá nhiều đặc trưng thời gian không cần thiết có thể làm tăng chiều dữ liệu và gây overfitting. Ngoài ra, với các đặc trưng mang tính chu kỳ, nếu encode theo dạng số nguyên thông thường (ví dụ Month = 1,...,12), mô hình có thể hiểu sai khoảng cách giữa các giá trị biên (tháng 12 và tháng 1).

Vì vậy, việc lựa chọn và mã hóa đặc trưng cần được thực hiện có kiểm soát và dựa trên bản chất toán học của dữ liệu.

b. Triển khai chi tiết

Quy trình triển khai gồm các bước:

- Trích xuất các thành phần thời gian cơ bản.
- Trích xuất các đặc trưng có tính chu kỳ.
- Trích xuất các đặc trưng nhị phân chứa thông tin sự kiện.
- Trích xuất đặc trưng chứa thông tin "Time Elapsed Since Specific Events".

Trích xuất các thành phần thời gian cơ bản Đầu tiên, chúng em trích xuất những thông tin cơ bản nhất là những thông tin cơ bản nhất của dữ liệu, việc trích xuất trước những thông tin này cũng góp phần thuận lợi cho việc trích xuất những đặc trưng phức tạp về sau. Về kỹ thuật, ở đây em trích xuất từ `DatetimeIndex`. Các đặc trưng chúng em trích xuất và lý do:

- Year: Năm, thông tin cơ bản.
- Month: Tháng, dùng cho bước tạo đặc trưng có chu kỳ.
- Day: Ngày, thông tin cơ bản.
- DayOfWeek (0 = Monday, ..., 6 = Sunday): Ngày trong tuần dạng index, dùng cho bước tạo đặc trưng có chu kỳ.
- DayName: Ngày trong tuần dạng text, đặc trưng này em trích xuất cho mục đích trực quan hóa.
- Quarter: Quý trong năm, thông tin cơ bản.
- DayOfYear: Ngày trong năm, thông tin cơ bản.

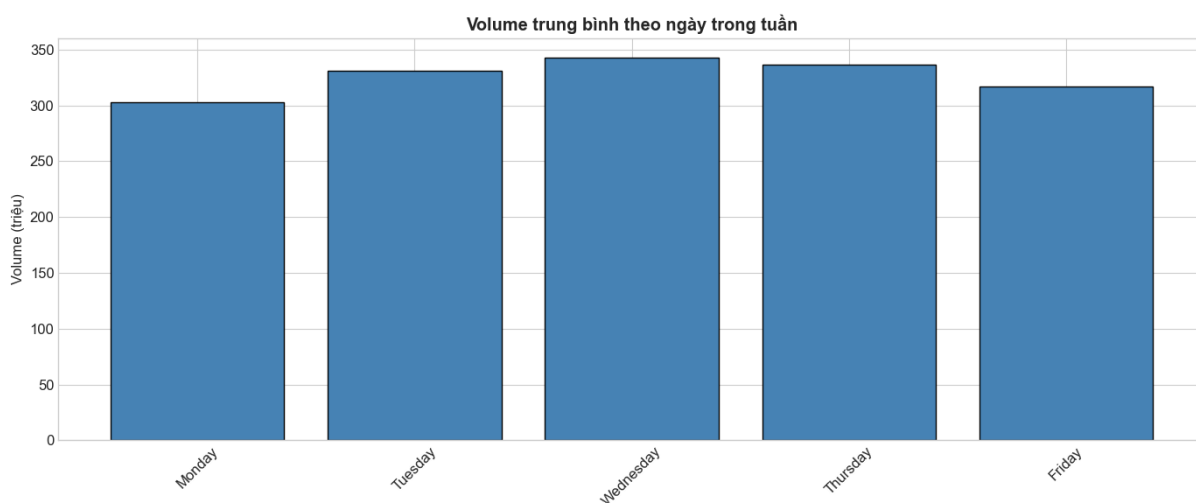


Figure 28: Biểu diễn Volume trung bình theo ngày trong tuần

Trích xuất các đặc trưng có tính chu kỳ Từ các thông tin thời gian đã trích xuất ở trên, em sẽ chọn ra những đặc trưng có tính chu kỳ để trích xuất. Các đặc trưng lựa chọn: Month (tháng), DayOfWeek (ngày trong tuần) và DayOfYear (ngày trong năm).

Chúng em sử dụng biến đổi:

$$x_{sin} = \sin\left(\frac{2\pi x}{T}\right), \quad x_{cos} = \cos\left(\frac{2\pi x}{T}\right)$$

Trong đó T là chu kỳ tương ứng (12 cho tháng, 7 cho ngày trong tuần, 365 cho ngày trong năm).

Các đặc trưng được tạo gồm:

- Month_sin, Month_cos
- DayOfWeek_sin, DayOfWeek_cos
- DayOfYear_sin, DayOfYear_cos

Sau khi khởi tạo, em sẽ dùng một biểu đồ để kiểm tra tính chu kỳ của những đặc trưng mới này.

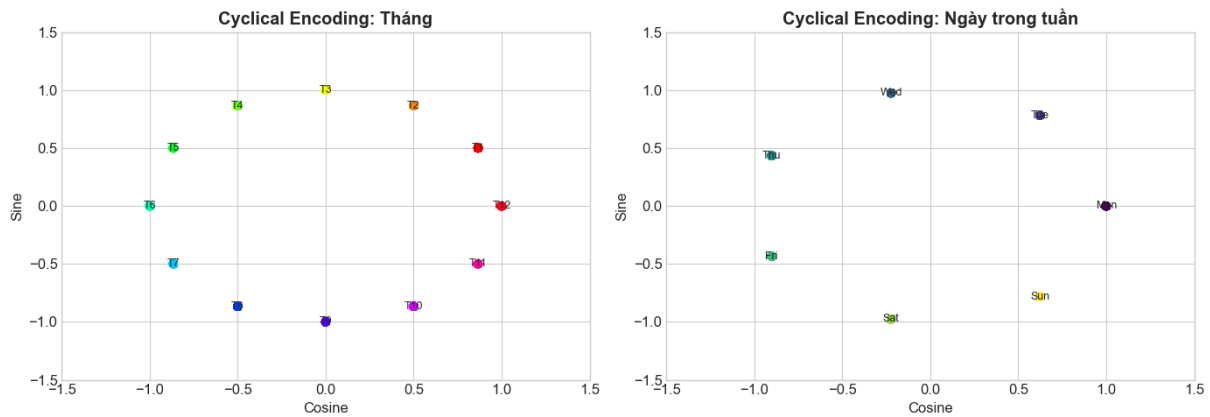


Figure 29: Biểu diễn Tháng trong năm, Ngày trong tuần

Trích xuất các đặc trưng nhị phân chứa thông tin sự kiện Để mô hình nắm bắt thông tin về sự kiện, em sẽ tạo các đặc trưng sau:

- IsMonday: Thứ Hai (0/1)
- IsFriday: Thứ Sáu (0/1)
- IsMonthStart: Ngày đầu tháng (0/1). Em không phân tích cuối tháng do cuối tháng và đầu tháng gần nhau(dữ liệu có tính chu kỳ và xu hướng).
- IsQuarterEnd: Ngày cuối Quý (0/1). Em không phân tích đầu Quý với lý do tương tự đặc trưng IsMonthStart.
- IsYearEnd: Ngày cuối năm (0/1).

Đề bài có yêu cầu xem xét holiday và weekend. Tuy nhiên, do dữ liệu chỉ bao gồm các ngày giao dịch thực tế (dữ liệu ở những ngày này trống, chúng em đã xử lý và giải thích ở 5.2.2 *Handling Time Gaps*), chúng em xin phép không khai thác thêm holiday hoặc weekend indicator trong phần này.

Trích xuất đặc trưng chứa thông tin "Time Elapsed Since Specific Events"

Để đáp ứng yêu cầu trích xuất khoảng thời gian kể từ một sự kiện cụ thể, chúng em xây dựng đặc trưng:

- DaysSinceMonthStart

Biến này được tính bằng số ngày kể từ ngày đầu tháng hiện tại:

$$\text{DaysSinceMonthStart} = \text{Date} - \text{FirstDayOfMonth}$$

Đặc trưng này giúp mô hình nắm bắt được vị trí tương đối của ngày trong tháng, từ đó có thể học các pattern liên quan đến đầu hoặc cuối chu kỳ tháng.

c. Kết quả và Nhận xét

- Trên thực tế, để tránh overfitting, nặng chi phí tính toán và đa cộng tuyến, chúng ta sẽ không cần trích xuất nhiều đặc trưng đến vậy.
- Để lựa chọn ra đặc trưng phù hợp cho bài toán, cần có thêm các bước đánh giá. Cho các đặc trưng này, em sẽ dùng correlation matrix để kiểm tra mức độ tương quan tuyến tính.

bỏ cái bảng tương quan vô r nhận xét cha nhận xét mẹ gì đó cho xong

4.2.4 Resampling Time Series Data

a. Cơ sở và lý do thực hiện

Lý do cần thực hiện

- Giảm nhiễu ngắn hạn do biến động theo ngày.
- Quan sát xu hướng dài hạn rõ ràng hơn.
- Phân tích tác động của các tần suất khác nhau lên đặc tính thống kê của chuỗi.
- So sánh các hàm aggregation khác nhau để lựa chọn cách tổng hợp phù hợp với dữ liệu tài chính.

Những hạn chế và rủi ro có thể phát sinh

- Làm mất thông tin chi tiết ở cấp độ ngày khi downsampling.
- Việc lựa chọn hàm tổng hợp không phù hợp có thể làm sai lệch đặc trưng dữ liệu (ví dụ dùng sum cho giá).
- Số lượng quan sát giảm đáng kể khi chuyển sang monthly.
- Upsampling yêu cầu nội suy và không tạo thêm thông tin thực tế.

Do dataset là dữ liệu giá cổ phiếu theo ngày, việc upsampling không được ưu tiên vì chỉ tạo giá trị nội suy mà không bổ sung thông tin giao dịch thực tế. Do đó, trọng tâm phân tích là downsampling.

b. Triển khai chi tiết

Quy trình thực hiện:

- Đầu tiên, chúng em sẽ thực hiện Downsampling dữ liệu từ theo ngày xuống theo tuần và theo tháng dựa trên OHLCV.
- Sau đó, chúng em sẽ so sánh hai tần suất Downsampling này.
- Cuối cùng sẽ đi vào chi tiết phân tích những hàm aggregation sử dụng.

Downsampling dữ liệu Trong quá trình tìm hiểu dữ liệu dạng stock để làm bài, em được làm quen với một khái niệm mới - OHLCV. Vì vậy, trước khi bước vào phần so sánh các aggregation function, em xin phép được chọn aggregation cho các feature theo những gì em tìm hiểu được.

Sử dụng hàm `resample()` của pandas với tham số:

- 'W' cho weekly.
- 'M' cho monthly.

Các hàm aggregation được lựa chọn dựa trên ý nghĩa của OHLCV:

- **Open:** dùng `first()` để lấy giá mở cửa đầu tuần/tháng.
- **High:** dùng `max()` để lấy giá cao nhất trong kỳ.
- **Low:** dùng `min()` để lấy giá thấp nhất trong kỳ.
- **Close:** dùng `last()` để lấy giá đóng cửa cuối kỳ.
- **Volume:** dùng `sum()` vì đây là tổng khối lượng giao dịch.

Sau đó, các thành phần được ghép lại bằng `pd.concat()` để tạo thành DataFrame weekly và monthly hoàn chỉnh.

So sánh tần suất Downsampling Để đánh giá ảnh hưởng của tần suất, chúng em so sánh biến `Close` trong giai đoạn 2019-2020. Ở đây, em chọn giai đoạn 2019-2020 do quan sát dữ liệu thấy đồ thị trong giai đoạn này phù hợp để khảo sát biến động.

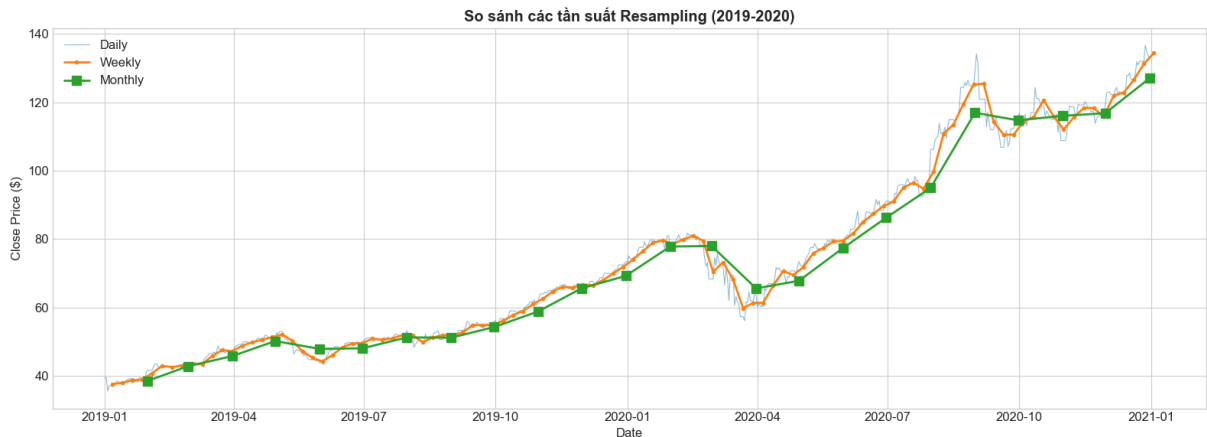


Figure 30: So sánh các tần suất Resampling (2019-2020)

Quan sát từ biểu đồ:

- Chuỗi gốc (được chú thích: daily) thể hiện rõ từng dao động của thị trường.
- Trong khi đó, chuỗi được downsampling theo tuần (được chú thích: weekly) và theo tháng (được chú thích: monthly) lần lượt mượt hơn và mượt hơn rất nhiều. Điều này cho thấy biên độ dao động ngắn hạn bị triệt tiêu dần khi tần suất giảm.
- Và quan trọng nhất là qua biểu đồ, ta biết được bước resampling đã được thực hiện thành công.

So sánh các hàm aggregation Chúng em chọn dùng tần suất Downsampling theo tháng và biến `Close` để khảo sát các hàm aggregation.

Các hàm được sử dụng gồm: `mean`, `median`, `min`, `max`, `sum`, `std`.

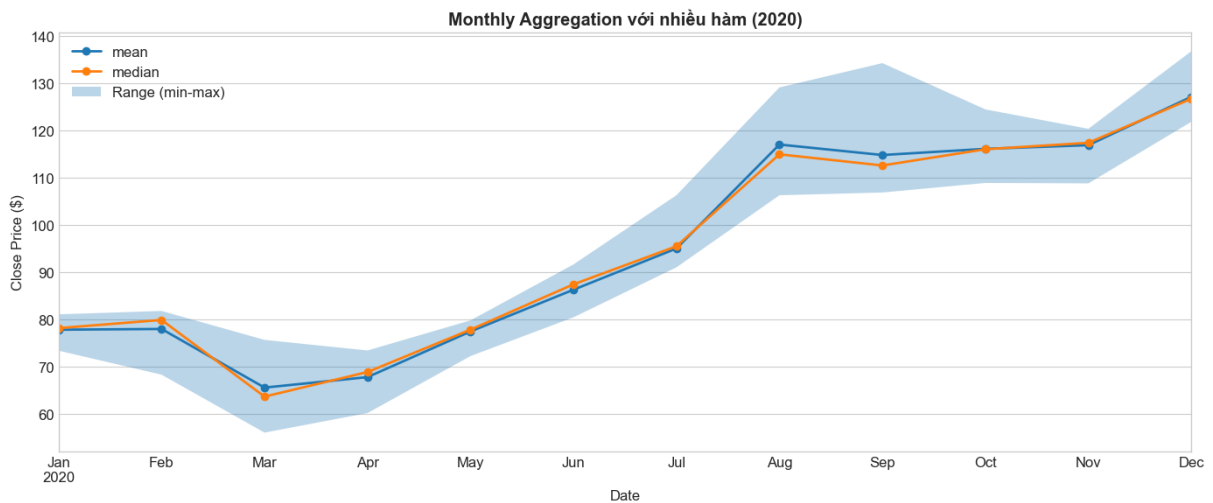


Figure 31: Khảo sát các hàm aggregation (2020)



Figure 32: Khảo sát hàm aggregation Sum (2020)

Kết quả cho thấy:

- **Mean** và **Median** khá tương đồng trong phần lớn các tháng, cho thấy phân phối giá không quá lệch.
- Khoảng **min-max** phản ánh mức độ biến động trong tháng, giãn rộng rõ ở một số tháng (tháng 8, 9).
- **Std** cho biết mức độ dao động của giá trong từng tháng, tức là giá thay đổi nhiều hay ít so với mức trung bình.
- **Sum** phụ thuộc vào số ngày giao dịch trong tháng, do đó không phù hợp để phân tích xu hướng giá.

c. Kết quả và Nhận xét

Từ toàn bộ quá trình resampling, có thể rút ra các nhận định sau:

- Khi muốn thực hiện bước Resampling nói chung hay Downsampling, Upsampling nói riêng cần phải xem xét về đặc tính dữ liệu cũng như nhu cầu của người thực hiện là gì. Vì 2 hướng resampling này sẽ dùng cho 2 mục đích hoàn toàn khác nhau.
- Việc lựa chọn aggregation function cần được lựa chọn dựa trên đặc tính của từng đặc trưng.

4.2.5 Lag Features

a. Cơ sở và lý do thực hiện

Lý do cần thực hiện

- Lag features là nền tảng của nhiều mô hình dự báo như AR, ARIMA và các mô hình Machine Learning.
- Giúp mô hình học được động học biến động giá thay vì chỉ dựa trên giá trị tại thời điểm hiện tại.

Những hạn chế và rủi ro có thể phát sinh

- Tạo lag làm xuất hiện giá trị thiếu (NaN) ở đầu chuỗi.
- Nếu tạo quá nhiều lag có thể làm tăng chiều dữ liệu và gây overfitting.
- Cần đảm bảo không sử dụng thông tin tương lai (tránh data leakage).

b. Triển khai chi tiết

Tạo Lag Features Chúng em sử dụng hàm `shift(k)` của pandas để tạo các đặc trưng:

- `Close_lag1 = Close.shift(1)`
- `Close_lag2 = Close.shift(2)`
- `Close_lag7 = Close.shift(7)`

Việc lựa chọn các độ trễ ngắn (1, 2 ngày) nhằm nắm bắt ảnh hưởng tức thời, trong khi lag 7 ngày giúp kiểm tra khả năng tồn tại chu kỳ theo tuần giao dịch.

Sau khi tạo lag, các giá trị NaN ở đầu chuỗi được loại bỏ bằng `dropna()` để đảm bảo dữ liệu sạch trước khi sử dụng cho mô hình.

Rolling Window Statistics Bên cạnh lag features, chúng em tạo thêm:

- Moving Average (rolling mean)
- Rolling Standard Deviation

Sử dụng hàm:

- `rolling(window=k).mean()`
- `rolling(window=k).std()`

Cửa sổ được chọn (ví dụ 7 hoặc 14 ngày) nhằm:

- Làm mượt dữ liệu (moving average).
- Đo mức độ dao động giá trong một khoảng thời gian cố định (rolling std).

Difference Features Chúng em tạo thêm đặc trưng sai phân:

$$\text{Diff}_t = \text{Close}_t - \text{Close}_{t-1}$$

hoặc sử dụng `diff()` trong pandas.

Mục đích của difference:

- Giảm xu hướng (trend) trong dữ liệu.
- Hỗ trợ quá trình làm cho chuỗi gần stationarity hơn.
- Là cơ sở cho các mô hình như ARIMA.

Thảo luận về Autocorrelation và lựa chọn Lag Dựa trên đặc tính của dữ liệu giá cổ phiếu:

- Autocorrelation thường cao ở các độ trễ nhỏ (lag 1, lag 2).
- Giảm dần khi độ trễ tăng.

Do đó, chúng em ưu tiên chọn các lag ngắn để:

- Tránh tăng chiều dữ liệu không cần thiết.
- Hạn chế nguy cơ overfitting.
- Giữ tính đơn giản cho bài toán preprocessing.

c. Kết quả và Nhận xét

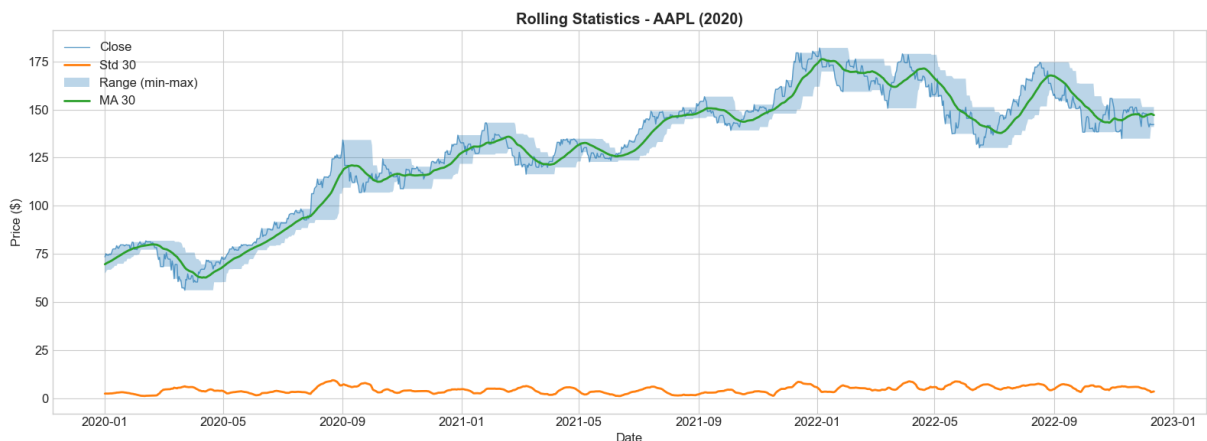


Figure 33: Rolling Statistics - AAPL (2020)

- Đường Moving Average mượt hơn so với chuỗi gốc, cho thấy tác dụng làm giảm nhiễu ngắn hạn.
- Rolling Standard Deviation phản ánh rõ những giai đoạn giá dao động mạnh hoặc ổn định.
- Ba đường (Close gốc, Moving Average, Rolling Std) thể hiện đúng đặc tính kỳ vọng của từng phép biến đổi, xác nhận rằng việc tạo rolling window statistic đã được thực hiện chính xác.

5 Đánh giá chung và thảo luận

Trong báo cáo này, chúng em đã thực hiện tiền xử lý trên bốn dạng dữ liệu quan trọng gồm ảnh, dữ liệu bảng, văn bản và chuỗi thời gian. Mỗi loại dữ liệu có đặc trưng cấu trúc riêng nên yêu cầu các kỹ thuật tiền xử lý khác nhau, nhưng đều hướng tới mục tiêu là làm sạch, chuẩn hóa và biểu diễn dữ liệu theo dạng phù hợp hơn cho các bước sau trong quy trình Khoa học dữ liệu. Với dữ liệu ảnh, các bước như resize, chuẩn hóa pixel và phát hiện biên giúp làm rõ đặc trưng hình dạng và giảm nhiễu; đồng thời thực nghiệm cho thấy lựa chọn phương pháp nội suy khi thay đổi kích thước ảnh tạo ra sự đánh đổi giữa độ mịn và độ sắc nét. Với dữ liệu bảng, xử lý giá trị thiếu, chuẩn hóa thang đo và mã hóa biến phân loại là điều kiện để mô hình hoạt động ổn định và tránh sai lệch. Đối với dữ liệu văn bản, pipeline gồm tokenization, loại bỏ stop-word có chọn lọc, lemmatization và vector hóa cho thấy tầm quan trọng của việc chuyển đổi ngôn ngữ tự nhiên thành biểu diễn số. Với dữ liệu chuỗi thời gian, các bước như parsing thời gian, resampling, xử lý missing và tạo lag features giúp trích xuất thông tin xu hướng và mùa vụ, từ đó hỗ trợ mô hình dự báo.

Mặc dù bốn loại dữ liệu khác biệt về bản chất, nhưng cùng chung nguyên tắc tiền xử lý như làm sạch dữ liệu, chuẩn hóa thang đo và giảm nhiễu nhằm cải thiện khả năng hội tụ và độ ổn định của mô hình sau này. Quan trọng hơn, chúng em nhận thấy tiền xử lý không thể áp dụng một cách máy móc mà cần dựa trên đặc điểm dữ liệu và mục tiêu bài toán cụ thể, bởi mỗi lựa chọn đều tồn tại trade-off giữa chi phí tính toán, độ phức tạp và hiệu năng mô hình.

6 Lời cảm ơn

Chúng em xin bày tỏ lòng biết ơn và cảm ơn đến Thầy Lê Hoài Bắc, Thầy Lê Nhựt Nam, Thầy Võ Nhật Tân vì sự hỗ trợ và chỉ dẫn của thầy trong quá trình thực hiện bài lab.

Bên cạnh đó, chúng em cũng xin ghi nhận sự hỗ trợ của các công cụ trí tuệ nhân tạo, bao gồm ChatGPT, Gemini trong việc gợi ý, hướng dẫn tìm kiếm tài liệu và chuẩn bị cấu trúc report cũng như tổng kết kiến thức.

Mặc dù bài làm đã được đầu tư và thực hiện một cách nghiêm túc, vẫn không tránh khỏi một số thiếu sót hoặc hiểu chưa đầy đủ do hạn chế và kiến thức, kinh nghiệm. Em mong nhận được sự góp ý để có thể hoàn thiện trong các bài khác trong tương lai.

7 Tài liệu tham khảo

- [1] M. Lin, Q. Chen, and S. Yan, “Network In Network.” <https://arxiv.org/pdf/1409.1556>, 2014. arXiv preprint arXiv:1409.1556.
- [2] W. contributors, “Grayscale.” <https://en.wikipedia.org/wiki/Grayscale>, 2026. Accessed: 2026-03-01.
- [3] TensorFlow, “Basic classification: Classify images of clothing.” <https://www.tensorflow.org/tutorials/keras/classification?hl=vi>, 2026. TensorFlow official tutorial, accessed 2026-03-01.
- [4] OpenCV Development Team, “Canny edge detector tutorial.” https://docs.opencv.org/4.x/da/d5c/tutorial_canny_detector.html, 2026. OpenCV 4.x documentation, accessed 2026-03-01.
- [5] FUNiX, “So sánh bộ công cụ ngôn ngữ tự nhiên nltk và spacy.” <https://funix.edu.vn/chia-se-kien-thuc/so-sanh-bo-cong-cu-ngon-ngu-tu-nhien-nltk-va-spacy/>, 2024. Truy cập: 2026-03-01.
- [6] C. Uit, “Nlp: Stemming và lemmatization.” <https://chienuit.wordpress.com/2016/05/03/nlp-stemming-va-lemmatization/>, 2016. Truy cập: 2026-03-01.